

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Wang, Tongli. "Topic 2.1. Introduction to Climate Data and Climate Models." *Climatic and Ecological Modelling for Adaptive Forest Applications*. Adaptation Learning Network.

<https://pressbooks.bccampus.ca/climatemodellingforestadaptation/chapter/topic-2-1-introduction-to-climate-models/>

- **Required pages:** The whole web page
- **Estimated reading time:** 1 hour

Lesson 1 - Climate Data

December 25, 2025

MODULE 7 | LESSON 1

1 Climate Data

Reading Time	60 minutes
Prior Knowledge	Basic Python
Keywords	climate data, point data, gridded data, GCM, Temporal resolution

1.1 1. Introduction

In this lesson, we are going to explore climate data. Climate change has been a popular issue today since it will have a significant impact on the environment we are living in. Climate change is happening now, and we need to learn to adapt to it. We can do this in the field of finance by studying and monitoring the potential risks of climate change on corporations and financial activities. Incorporating climate data and analyses will offer extra insight into the risk management of these activities.

1.2 2. Climate Data and Financial Analysis

1.2.1 2.1 What is Climate Data?

Climate data is the information collected on Earth's land, seas, and the surrounding atmosphere (EPRI). As shown in Figure 1, we can classify climate data into five categories: temperature, wind, hydrologic, precipitation, and solar. **Figure 1. Climate Data Categories and Examples**

Category	Example
Temperature	Air temperature, extreme temperature, humidity
Wind	Wind Speed, Wind Direction
Hidrologic	Riverflow, Runoff
Precipitation	Rainfall, Snowfall
Solar	Cloud Cover, Radiation

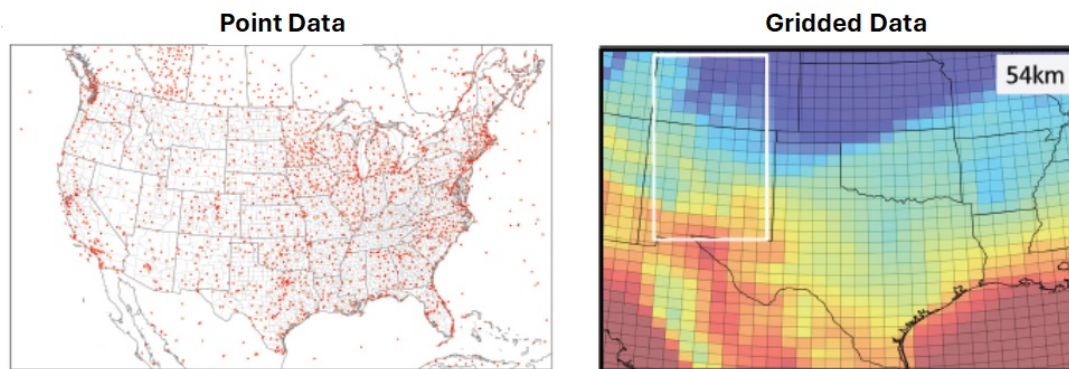
Source: EPRI

2.2 Impact of Climate Data on Financial Analysis Climate change has a significant impact on the opportunities and risks associated with investment activities, financial operations, and business strategies (Intergovernmental Panel on Climate Change [IPCC]). Climate data and climate analysis are key to monitoring climate change. Risks like severe weather events can cause damage to physical assets and disrupt business activities. Shifting to using clean energy provides new investment opportunities as the traditional energy sector faces uncertainty. Climate data can be incorporated into financial analysis to provide a new perspective on how to handle financial issues. By analyzing climate data, we can manage various natural risks, identify new business opportunities, and continue business under emerging climate-related regulations.

1.3 3. Characteristics of Climate Data

1.3.1 3.1 Data Coverage

There are two ways to present climate data: **point data** and **gridded data**. **Point data** is the climate data representative of a specific location and is collected from surface stations scattered around the globe. Since a station is located at a specific point, the data collected will represent the climate conditions for that specific location. Surface stations are distributed unevenly; as such, point data may be unavailable for certain parts of the globe. We will talk about this issue later. **Gridded data** is the climate data representative of a specific area on Earth. In order to collect Earth data systematically, scientists divide the globe into many squares to create a grid system, like CRS, which we introduced in the lesson on geospatial data. Then, climate data is collected for each squared area. Figure 2 demonstrates the concept of data covering areas of point data and gridded data. **Figure 2. Data Covering Areas of Point Data and Gridded Data**

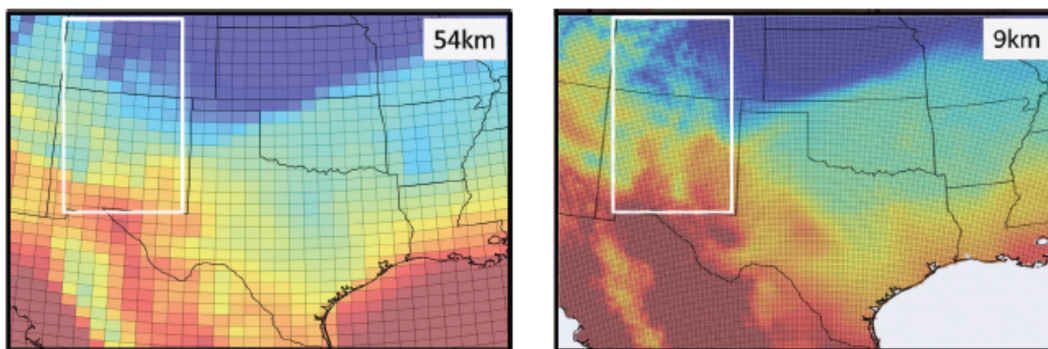


Source:

EPRI The grid system varies according to the size of the square. The length of the square can range from 9km to 111km. The shorter the length of the square, the more squares can cover the globe and the more data can be collected. This results in higher resolution of the data. We can also say this data has higher spatial resolution. Figure 3 demonstrates examples of different spatial resolutions. **Figure 3. Low Spatial Resolution vs. High Spatial Resolution**

Lower Resolution With Square Length 54km

Higher Resolution With Square Length 9km



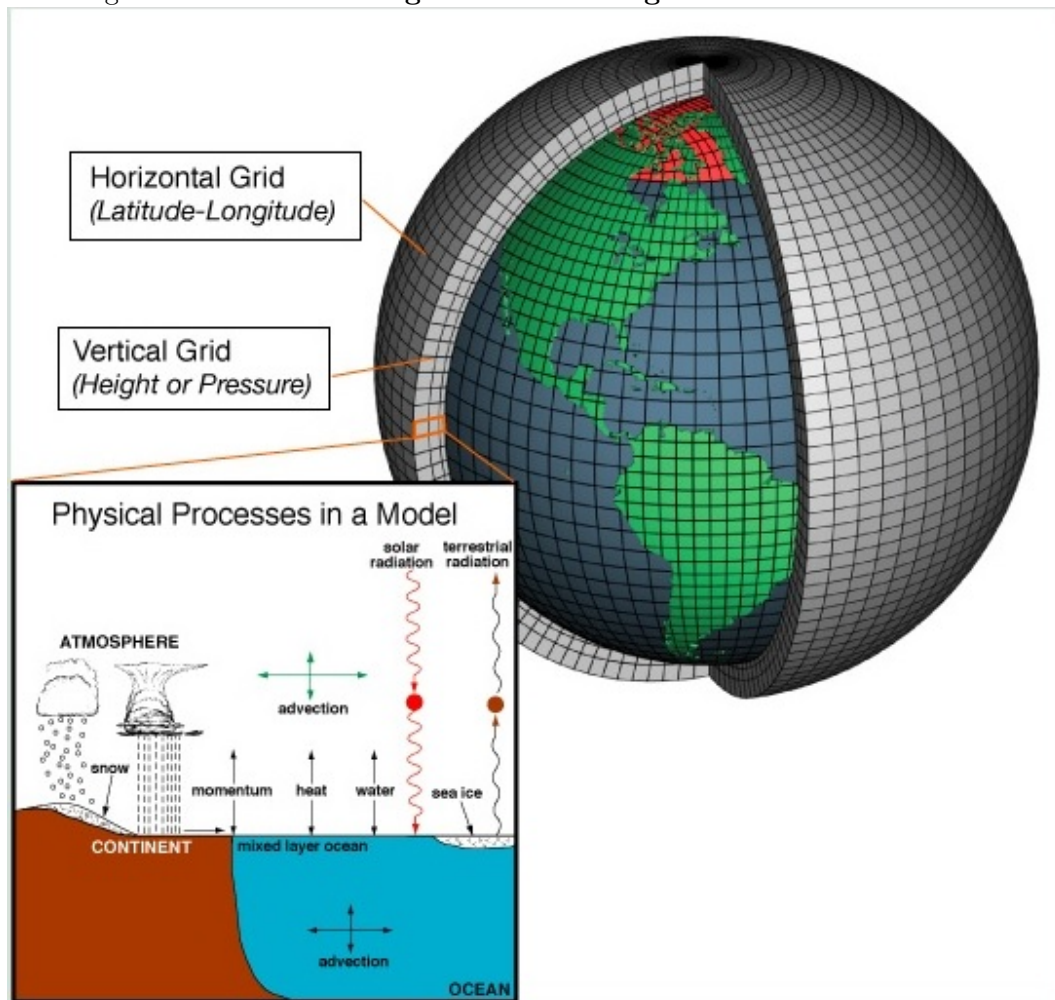
Source:

EPRI Both point data and gridded data can be collected with different time frequencies. The common frequencies are hourly, daily, seasonal, and annual data. The more frequently data is collected, the more data is available. We usually call the data collected with higher frequencies as having higher **temporal resolution**. ### **3.2 Data Sources** There are two sources for climate data: **direct observation** and **reanalysis**. **Direct observation** involves collecting climate data through surface station networks and satellite observation. A station records the precipitation, air temperature, and other climate data at a specific location. Direct observation data from a surface station is also called point data as described above. The earliest historical direct observation data can go back to around the 1850s. There can be some issues with direct observation data. The first one is a data gap. If a station's monitoring sensor breaks down, there will be missing data. The second issue is unevenly distributed observations. In the last section, we mentioned that surface stations are not evenly distributed around the globe. Currently, there are more stations set up in Europe and North America. Observation data outside of these two regions are sparse. Also, the data frequency or temporal range can be different by data type. Hence, direct observation data alone cannot provide us with a comprehensive view of the global climate system. **Reanalysis** is a method to combine historical direct observation data with short-range forecasted data from previous observations to fill in data gaps and get the best view of the current Earth system. With reanalysis, we can get complete actual-simulated-combined historical global climate data, and the data will also be consistent in time.

1.4 4. Climate Projection Data

In the last section, we talked about the data sources for historical climate data. Earth scientists have also been working closely to create **projected** climate data into the next 30 years and even further out. Projected climate data will provide us with a forward-looking view for climate change and help us plan to adapt to future changes to our living environment. There are two important components for generating climate projection data: **climate models** and **climate scenarios**.

4.1 Climate Models A climate model contains a collection of physics principles and mathematical equations to represent the Earth system. The model usually contains thousands of lines of codes and requires a supercomputer to run it. There are climate models with different coverages: some climate models only focus on specific regions of the Earth, while some climate models cover the whole planet. One group of popular climate models is the general circulation models or global climate models (GCMs). **Global climate models (GCM)** are 3D climate models that cover the interactions between the atmosphere, land, and ocean on the whole Earth. 3D means the model includes latitude, longitude, and height (pressure). GCMs divide the Earth into grid cells and model on each cell and its interactions with neighboring grid cells. Figure 4 demonstrates the modeling structure of GCM. **Figure 4. Modeling Structure of Global Climate Mod-**

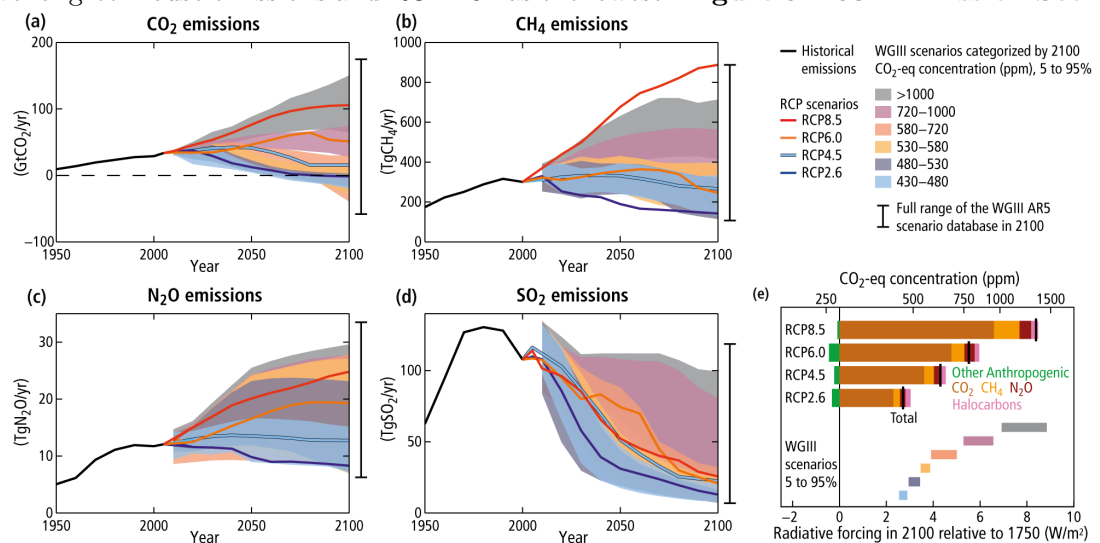


els

Source:

NOAA The key input variables to a GCM are external factors that can change the amount of energy from the Sun that is absorbed on Earth or will stay in the atmosphere. Some of the input variables

are changes in the Sun's energy emissions and changes in greenhouse emissions. The model can output thousands of variables in different timeframes (hourly, daily, seasonal, and annual) to provide a complete view of the Earth climate system. **4.2 Climate Scenarios** Once a GCM is built, we can use the model to simulate the future Earth climate system and obtain projected climate data. To conduct a simulation, we need to provide climate scenarios as inputs to the model. There are several GCMs built by different institutions around the globe. In order for the projected outputs from different GCMs to be comparable, the Intergovernmental Panel of Climate Change (IPCC) from the United Nations has created standardized climate scenarios for climate projection. The most commonly used scenarios are **representative concentration pathways (RCP)**. There are four pathways showing different levels of greenhouse emissions, air pollution, and other external factors in the 21st century. They are RCP8.5, RCP6.5, RCP4, and RCP2.6. RCP8.5 has the highest level of greenhouse emissions and RCP2.6 has the lowest. **Figure 5. RCP Emission Scenarios**



narios

Source: IPCC When generating projected climate data, it is recommended to use several different GCMs (ensemble method) with climate scenarios to simulate future climate changes and use the averages of output data from different GCMs as climate projected data.

1.5 5. Application with Python

In this section, we're going to demonstrate how to retrieve climate data from the **NOAA Physical Science Laboratory (PSL)** using Python. We will also introduce a data format called NetCDF4 that is commonly used to store climate data. Then, we will use Python to process and clean the data to be ready for analysis. **5.1 Climate Data from NOAA PSL** NOAA is short for **National Oceanic and Atmospheric Administration**, a U.S. governmental organization in charge of environmental intelligence. The Physical Science Laboratory (PSL) is a division in NOAA responsible for weather, climate, and hydrologic research. PSL houses a large amount of gridded climate data that is open to public scientists. You can find more information about available data from NOAA PSL [here](#). **5.2 NetCDF Data Format** **Network Common Data Form (NetCDF)** is a popular data format to store gridded climate data. NetCDF is a multidimensional and array-based data format. NetCDF also contains its metadata and data attribute information. Hence, it is also called a "self-describing" data format. There are several versions of NetCDF data formats, but the latest version is NetCDF4. This is the data format currently used for NOAA PSL gridded climate data. We will discuss more properties of the NetCDF data format in the following

example. **5.3 U.S. Daily Precipitation Data - Data Retrieve** We will use U.S. daily precipitation data for this demonstration. There has been tremendous progress made by scientists for global precipitation analysis for the last two decades (Xie et al.). This section will give you an overview of how to collect the data and analyze it. You can read the description of the dataset [here](#). We are going to pull the data from 2011 to 2014. The Python example is based on Ali Ahmadalipour's Google Colab notebook example.

```
[1]: # xarray library is used to handle NetCDF file
# In order to use the latest xarray library, we need to use a specific version
# of importlib-metadata library
!pip install importlib-metadata==4.13.0
```

Requirement already satisfied: importlib-metadata==4.13.0 in
/usr/local/lib/python3.11/site-packages (4.13.0)

Requirement already satisfied: zipp>=0.5 in /usr/local/lib/python3.11/site-
packages (from importlib-metadata==4.13.0) (3.21.0)

WARNING: Running pip as the 'root' user can result in broken permissions

and conflicting behaviour with the system package manager. It is recommended to
use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip
available: 22.3.1 -> 25.3

[notice] To update, run:
pip install --upgrade pip

```
[2]: # Import libraries for this demonstration
import glob
import matplotlib.pyplot as plt
import urllib.request
import xarray as xr
import pandas as pd
```

```
[ ]: # Retrieve data from NOAA PSL. Use for loop to pull data from 2011 to 2014
for yr in range(2011,2015):
    url = f'https://downloads.psl.noaa.gov/Datasets/cpc_us_precip/RT/precip.V1.
    0.{yr}.nc'
    savename = url.split('/')[-1]
    urllib.request.urlretrieve(url,savename)
```

1.5.1 5.4 Data Inspection

```
[3]: # use open_dataset function from xarray to open data files for 2011 and assign
# file names
ds2011 = xr.open_dataset('precip.V1.0.2011.nc')
ds2011
```

```
[3]: <xarray.Dataset>
Dimensions: (lat: 120, lon: 300, time: 365)
Coordinates:
  * lat      (lat) float32 20.12 20.38 20.62 20.88 ... 49.12 49.38 49.62 49.88
  * lon      (lon) float32 230.1 230.4 230.6 230.9 ... 304.1 304.4 304.6 304.9
  * time     (time) datetime64[ns] 2011-01-01 2011-01-02 ... 2011-12-31
Data variables:
  precip    (time, lat, lon) float32 ...
Attributes:
  title:      CPC Unified Gauge-Based Analysis of Daily Precipitation o...
  Conventions: COARDS
  description: Gridded daily Precipitation
  platform:   Observations
  Comments:   Precipitation is accumulated from 12z of previous day to 1...
  history:    originally created RT starting 04/2010 by CAS from data o...
  dataset_title: CPC Unified Gauge-Based Analysis of Daily Precipitation o...
  References: http://www.psl.noaa.gov/data/gridded/data.unified.daily.c...
```

We presented the metadata for 2011 data in the above code. From the above metadata, we can see there are 3 dimensions: latitude, longitude, and time. There is one variable, precip, in the dataset. The NetCDF data format is a hierarchical data structure. The longitude and latitude dimensions identify the location of the daily precipitation on the grid. The time dimension identifies the time location of the array. Let's check out some data from a 2011 dataset.

```
[4]: # Check out the latitude values of first 4 observations from 2011 dataset
ds2011["precip"]["lat"].values[:4]
```

```
[4]: array([20.125, 20.375, 20.625, 20.875], dtype=float32)
```

```
[5]: # Check out the longitude values of first 4 observations from 2011 dataset
ds2011["precip"]["lon"].values[:4]
```

```
[5]: array([230.125, 230.375, 230.625, 230.875], dtype=float32)
```

```
[6]: # Check out the time values of first 4 observations from 2011 dataset
ds2011["precip"]["time"].values[:4]
```

```
[6]: array(['2011-01-01T00:00:00.000000000', '2011-01-02T00:00:00.000000000',
          '2011-01-03T00:00:00.000000000', '2011-01-04T00:00:00.000000000'],
          dtype='datetime64[ns]')
```

```
[7]: # Check attribution of 2011 dataset
ds2011.attrs
```

```
[7]: {'title': 'CPC Unified Gauge-Based Analysis of Daily Precipitation over CONUS RT
at PSD',
      'Conventions': 'COARDS',
      'description': 'Gridded daily Precipitation',
```

```

'platform': 'Observations',
'Comments': 'Precipitation is accumulated from 12z of previous day to 12z of day
stored',
'history': 'originally created RT starting 04/2010 by CAS from data obtained
from NCEP/CPC\n converted to unpacked chunked netCDF4 Aug 2014',
'dataset_title': 'CPC Unified Gauge-Based Analysis of Daily Precipitation over
CONUS',
'References':
'http://www.psl.noaa.gov/data/gridded/data.unified.daily.conus.rt.html'}

```

1.5.2 5.5 Data Slice

Now, let's try to pull some data points from the 2011 dataset and draw a graph. Since the whole dataset is for the U.S., let's focus on a city and use Boston as an example. The latitude of Boston is 42.36 and the longitude is -71.05. Let's first put this coordinate on a map to make sure this is the right one.

```

[8]: # Import a mapping library
import folium

```

```

[9]: # Create a map with the coordinates for Boston
latitude = 42.361145
longitude = -71.057083
map = folium.Map(location=[latitude, longitude], zoom_start=14,
↳control_scale=True)
folium.Marker(location=[latitude, longitude]).add_to(map)
map

```

```

[9]: <folium.folium.Map at 0x7065a13cde10>

```

```

[10]: # Slice the data for Boston. In this dataset, the longitude coordinate is
↳represented by the absolute value of data value + 180.
latitude = 42.361145
longitude = 71.057083 + 180
boston_precip = ds2011.precip.sel(lat=latitude, lon=longitude, method =
↳'nearest')
boston_precip

```

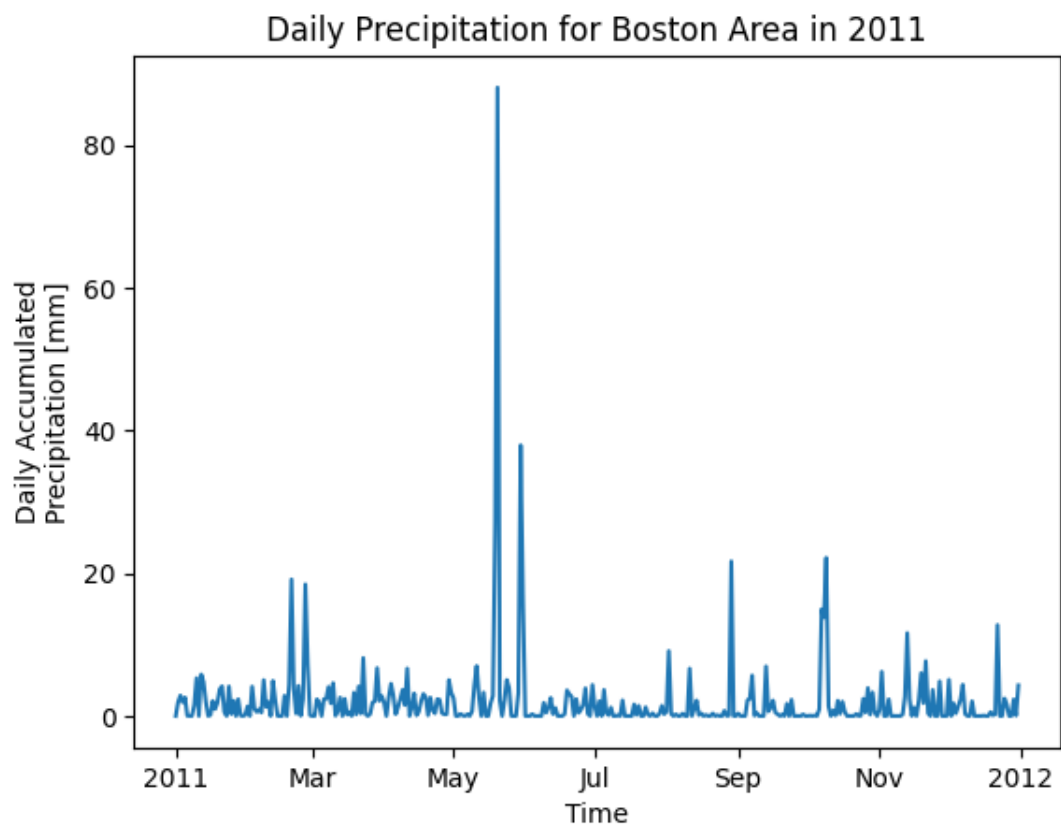
```

[10]: <xarray.DataArray 'precip' (time: 365)>
[365 values with dtype=float32]
Coordinates:
    lat      float32 42.38
    lon      float32 251.1
    * time    (time) datetime64[ns] 2011-01-01 2011-01-02 ... 2011-12-31
Attributes:
    long_name:    Daily Accumulated Precipitation
    valid_range:  [ 0. 700.]
    units:        mm

```

```
precision:      2
var_desc:       Precipitation
dataset:        CPC Unified Gauge-Based Analysis of Daily Precipitation ov...
level_desc:     Surface
statistic:      Daily Accumulation
parent_stat:    Observation
actual_range:   [ 0.      260.58218]
```

```
[11]: # Let's draw a line graph
      boston_precip.plot.line()
      plt.title("Daily Precipitation for Boston Area in 2011")
      plt.show()
```



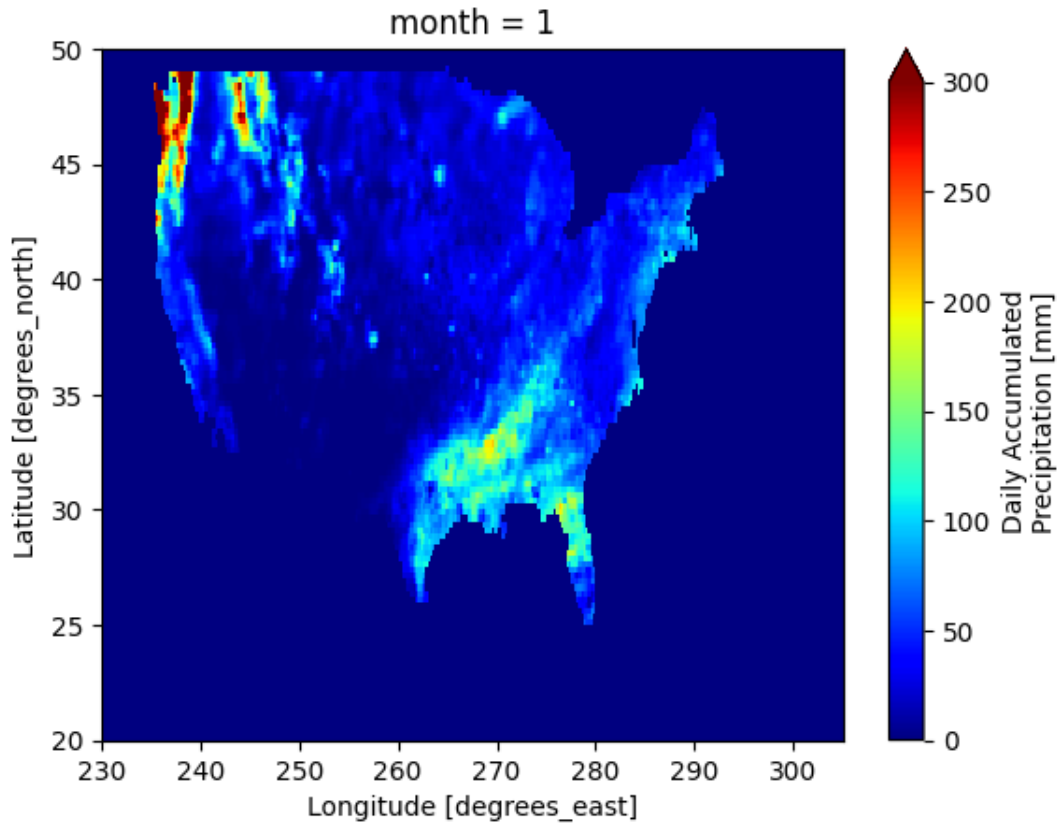
1.5.3 5.6 Data Aggregation

In this section, let's do some data aggregation. First, we'll use the 2011 dataset. It's a daily precipitation dataset. Let's aggregate to a monthly precipitation dataset and visualize it. The following Python code demonstrates how this is done.

```
[12]: # Aggregate 2011 dataset from daily to monthly and visualize it.
```

```
ds2011_precip_mon = ds2011.groupby('time.month').sum()  
ds2011_precip_mon.precip[0].plot(cmap='jet', vmax=300)
```

```
[12]: <matplotlib.collections.QuadMesh at 0x7065a1273bd0>
```



In the above data aggregation example, we rolled up daily precipitation data into monthly precipitation data and showed the amount of precipitation in January 2011 on a map. The next aggregation we would like to demonstrate is combining several files into one. At the start of this lesson, we downloaded data from 2011–2014 from NOAA PSL. The following code shows how to combine them together.

```
[13]: # Combine several datasets
```

```
ds2011_2014 = xr.open_mfdataset('precip.V1.0.*.nc', concat_dim='time',  
                                combine='nested')  
ds2011_2014
```

```
[13]: <xarray.Dataset>
```

```
Dimensions: (lat: 120, lon: 300, time: 1461)
```

```
Coordinates:
```

```

* lat      (lat) float32 20.12 20.38 20.62 20.88 ... 49.12 49.38 49.62 49.88
* lon      (lon) float32 230.1 230.4 230.6 230.9 ... 304.1 304.4 304.6 304.9
* time     (time) datetime64[ns] 2011-01-01 2011-01-02 ... 2014-12-31
Data variables:
    precip    (time, lat, lon) float32 dask.array<chunksize=(1, 120, 300),
meta=np.ndarray>
Attributes:
    title:      CPC Unified Gauge-Based Analysis of Daily Precipitation o...
    Conventions: COARDS
    description: Gridded daily Precipitation
    platform:    Observations
    Comments:    Precipitation is accumulated from 12z of previous day to 1...
    history:     originally created RT starting 04/2010 by CAS from data o...
    dataset_title: CPC Unified Gauge-Based Analysis of Daily Precipitation o...
    References:  http://www.psl.noaa.gov/data/gridded/data.unified.daily.c...

```

From the above metadata, we can see the data variable is precipitation (precip) and the coordinates are latitude, longitude, and time. The time dimension is daily frequency. The following chart from Python can give us an idea of what the metadata structure looks like.

```
[14]: ds2011_2014.precip.data
```

```
[14]: dask.array<concatenate, shape=(1461, 120, 300), dtype=float32, chunksize=(1,
120, 300), chunktype=numpy.ndarray>
```

From the above 3D chart, one rectangular shape consists of 36,000 (300 X 120) precipitation data points for a day. There are 1,461 days from 2011 to 2014. On the time dimension, we can roll up by month or year. The following example shows how to roll up the daily data into annual data.

```
[15]: data_temp = ds2011_2014.groupby("time.year").sum()
data_temp.precip.data
```

```
[15]: dask.array<concatenate, shape=(4, 120, 300), dtype=float32, chunksize=(1, 120,
300), chunktype=numpy.ndarray>
```

With a better understanding of the data structure, we can easily manipulate the dataset to get the different time aggregations we would like to have. In the next section, we'll show you how to turn this dataset into a pandas dataframe for future analysis work.

#5.7 Convert NetCDF data to dataframe and save as a .csv file. In this section, we are going to show you how to convert a NetCDF data into a data frame. Here is the code.

```
[16]: # Convert NetCDF to dataframe
ds2011_2014_df = ds2011_2014.to_dataframe()
ds2011_2014_df = ds2011_2014_df.dropna()
ds2011_2014_df.head()
```

```
[16]:          precip
lat    lon    time
```

25.125	278.875	2011-01-01	0.0
		2011-01-02	0.0
		2011-01-03	0.0
		2011-01-04	0.0
		2011-01-05	0.0

```
[17]: type(ds2011_2014_df)
```

```
[17]: pandas.core.frame.DataFrame
```

From the above code, we learned how to convert the NetCDF4 dataset to a pandas dataframe. This will help us in future analysis.

1.6 6. Conclusion

In this lesson, we first introduced what climate data is and then described its attributes. In the second half of the lesson, we demonstrated how to pull climate data from a government agency and process climate data for future analysis. This lesson only covered the most basic concepts and methods for exploring climate data. For those who are interested in this topic, there is more information online.

1.7 References

- Ahmadalipour, Ali. “Tutorial_Python_Geospatial.” April 2022. <https://colab.research.google.com/drive/1B7gFBSr0eoZ5IbsA0lY8q3XL8n-3BOn4>.
- EPRI. “Climate 101: Physical Climate Data, An Introduction to Climate Data and Implications to Electricity Sector.” Physical Climate Data 101. 2023.
- Intergovernmental Panel on Climate Change. “Chapter 15: Investment and Finance.” *IPCC Sixth Assessment Report*. <https://www.ipcc.ch/report/ar6/wg3/chapter/chapter-15/>
- Xie, Pingping, et al. “A Gauge-Based Analysis of Daily Precipitation over East Asia.” *Journal of Hydrometeorology*, vol. 8, no. 3, 2007, p. 607–626.

Matrix Normalization and Sparse Climate Data Compression

Reading Time 60 minutes

Prior Knowledge Basic Python

Keywords |climate data, matrix normalization, standardization, min-max scaling, robust scaling, temporal normalization, compressed sparse row format | | |

1. Introduction

In previous lessons, we learned the basics of climate data. In this lesson, we are going to learn how to manipulate climate data in a matrix form. The first step we will learn is to normalize matrix climate data. We will introduce three normalization methods: Z-score normalization, min-max scaling, and log transformation. We will also provide applications to demonstrate how to use these methods. Next, we will talk about sparse matrices. We will describe what a sparse matrix is and its issues in data analysis. Then, we'll introduce a method for handling a sparse matrix data. But first, let's introduce the dataset we'll use in this lesson.

2. Dataset

2.1 Financial Data

For this lesson, we will use both climate data and financial data.

For financial data, we will use the daily price of SPY, SPDR S&P 500 Index ETF Trust, from 2010 to 2023. SPY is the most widely traded ETF and is commonly used as the benchmark for the U.S. stock market's overall performance. Let's pull this data from Yahoo's website first.

```
In [1]: import pandas as pd
import requests
from io import StringIO
import datetime
import matplotlib.pyplot as plt
import numpy as np
import yfinance as yfin
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from scipy.sparse import csr_matrix
```

```
In [2]: # Pull S&P ETF price data
start = "2010-01-01"
end = "2023-12-31"
stocks = yfin.download(["SPY"], start, end)["Close"]
stocks.head()
```

```
/tmp/ipython-input-726302562.py:4: FutureWarning: YF.download() has changed argument
auto_adjust default to True
stocks = yfin.download(["SPY"], start, end)["Close"]
[*****100%*****] 1 of 1 completed
```

```
Out[2]:
```

	Ticker	SPY
	Date	
	2010-01-04	85.027962
	2010-01-05	85.253021
	2010-01-06	85.313049
	2010-01-07	85.673195
	2010-01-08	85.958290

```
In [3]: # Create an index for the stock dataframe
stocks.index = pd.to_datetime(stocks.index).strftime("%Y-%m-%d")
```

2.2 Climate Data

In this section, we are going to demonstrate how to pull climate data from the National Centers for Environmental Information. **The National Centers for Environmental Information (NCEI)** is a division of NOAA (National Oceanic and Atmospheric Administration) and maintains one of the world's largest repositories of environmental data. Its data includes climate data, ocean data, and satellite data.

We will extract some climate data from the **Global Historical Climatology-Daily (GHCN-Daily)** database of NCEI. GHCN-Daily combines daily climate data from approximately 30 different data sources. It collects data measurements from over 90,000 land-based stations worldwide. You can find more information about the GHCN-Daily database [here](#).

In our case, we will extract our climate data measured at the station in Central Park, New York City, New York, in the United States. The station id code is "USW00094728". You can find the list of all station identifiers for GHCN-Daily [here](#).

We are going to pull the following daily climate variables between 2010 and 2023:

PRCP: Precipitation (millimeters, mm)
SNOW: Snowfall (centimeter, cm)
TMAX: Daily Max Temperature (Degrees, Celsius)
TMIN: Daily Min Temperature (Degrees, Celsius)
AWND: Daily wind speed (Meters per second, m/s)
RHAV: Relative Humidity (%)

For a whole list of available climate variables in the GHCN-Daily database, go [here](#).

```
In [4]: #Access data through NCEI API

#Set parameters
dataset = "daily-summaries"           # this is the name for GHCN-Daily dat
vars = "PRCP, SNOW, TMIN, TMAX, AWND, RHAV" # climate variable symbols
station_id = "USW00094728"           # New York Central Park station ident
st_dt = "2010-01-01"                 # start date
end_dt = "2023-12-31"                 # end date

api_base = "https://www.ncei.noaa.gov/access/services/data/v1?"
req_params = "dataset=" + dataset + "&dataTypes=" + vars + "&stations=" + station_id
req_str = api_base + req_params
data = requests.get(req_str)
```

```
In [5]: #Convert the downloaded data into a pandas dataframe
s=str(data.content,'utf-8')
data_s = StringIO(s)
climate=pd.read_csv(data_s)
```

```
In [6]: climate.head()
```

```
Out[6]:
```

	STATION	NAME	DATE	AWND	PRCP	RHAV	SNOW	TMAX	TMIN
0	USW00094728	NY CITY CENTRAL PARK, NY US	2010- 01-01	1.5	0.8	83.0	0.0	4.4	0.6
1	USW00094728	NY CITY CENTRAL PARK, NY US	2010- 01-02	5.6	0.5	61.0	5.0	1.1	-8.3
2	USW00094728	NY CITY CENTRAL PARK, NY US	2010- 01-03	6.4	0.0	56.0	0.0	-5.6	-8.3
3	USW00094728	NY CITY CENTRAL PARK, NY US	2010- 01-04	4.8	0.0	56.0	0.0	-1.1	-7.2
4	USW00094728	NY CITY CENTRAL PARK, NY US	2010- 01-05	4.4	0.0	56.0	0.0	-1.1	-6.7

We now have both a financial dataset and a climate dataset, so let's merge the two datasets together using the common variables.

```
In [7]: climate_stock = climate.merge(stocks, left_on='DATE', right_on='Date', how = 'inner')
climate_stock.head()
```

```
Out[7]:
```

	STATION	NAME	DATE	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
0	USW00094728	NY CITY CENTRAL PARK, NY US	2010-01-04	4.8	0.0	56.0	0.0	-1.1	-7.2	85.027962
1	USW00094728	NY CITY CENTRAL PARK, NY US	2010-01-05	4.4	0.0	56.0	0.0	-1.1	-6.7	85.253021
2	USW00094728	NY CITY CENTRAL PARK, NY US	2010-01-06	4.3	0.0	56.0	0.0	1.1	-3.3	85.313049
3	USW00094728	NY CITY CENTRAL PARK, NY US	2010-01-07	3.9	0.0	63.0	0.0	2.8	-1.1	85.673195
4	USW00094728	NY CITY CENTRAL PARK, NY US	2010-01-08	3.4	1.3	70.0	15.0	0.6	-5.0	85.958290

The above dataset now includes both financial and climate data. We also know that different variables have different measurement units. In the following sections, we will talk about how to normalize these climate and financial variables for financial analysis.

3. Matrix Normalization for Climate Data

3.1 Why Do We Need Matrix Normalization?

A lot of data comes in matrix form, including climate data. For example, you might get climate data for a location in matrix form. In the climate data matrix, there can be a temperature column, a wind speed column, and a precipitation column. They represent different variables in the data matrix. There can be several rows in the data matrix to represent dates and their values in the temperature, wind speed, and precipitation variables. To be more specific, each row can be the daily average measurements of these variables.

This is a very structured data matrix. However, there are a few issues in this data matrix we need to address before using it for analysis.

1. Variables in the climate data matrix have different measurement units or scales. We cannot directly compare between these variables.
2. Variables in the climate data matrix have different data distributions. Many financial analysis techniques require normal distribution of the data.
3. Many financial analysis techniques also require data normalization as a prerequisite.

Our dataset clearly suffers from issue 1. In our dataset, there are 6 climate variables with 5 different measurement units. Once we add SPY variable into the dataset, we will have 7 variables with 6 different measurement units.

Our dataset also suffers from issue 2, where different variables have different distributions. Let's use the following graph to demonstrate this issue.

```
In [8]: # Show histogram for each variable to demonstrate issue 2
# Initialize a 3x3 charts
def hist_graph(dataset_x, var_list, row, col):
    fig, axes = plt.subplots(nrows=row, ncols=col, figsize=(8, 8))

    # Flatten the axes array (makes it easier to iterate over)
    axes = axes.flatten()

    dataset_y = dataset_x[var_list]
    # Loop through each column and plot a histogram
    for i, column in enumerate(dataset_y.columns):

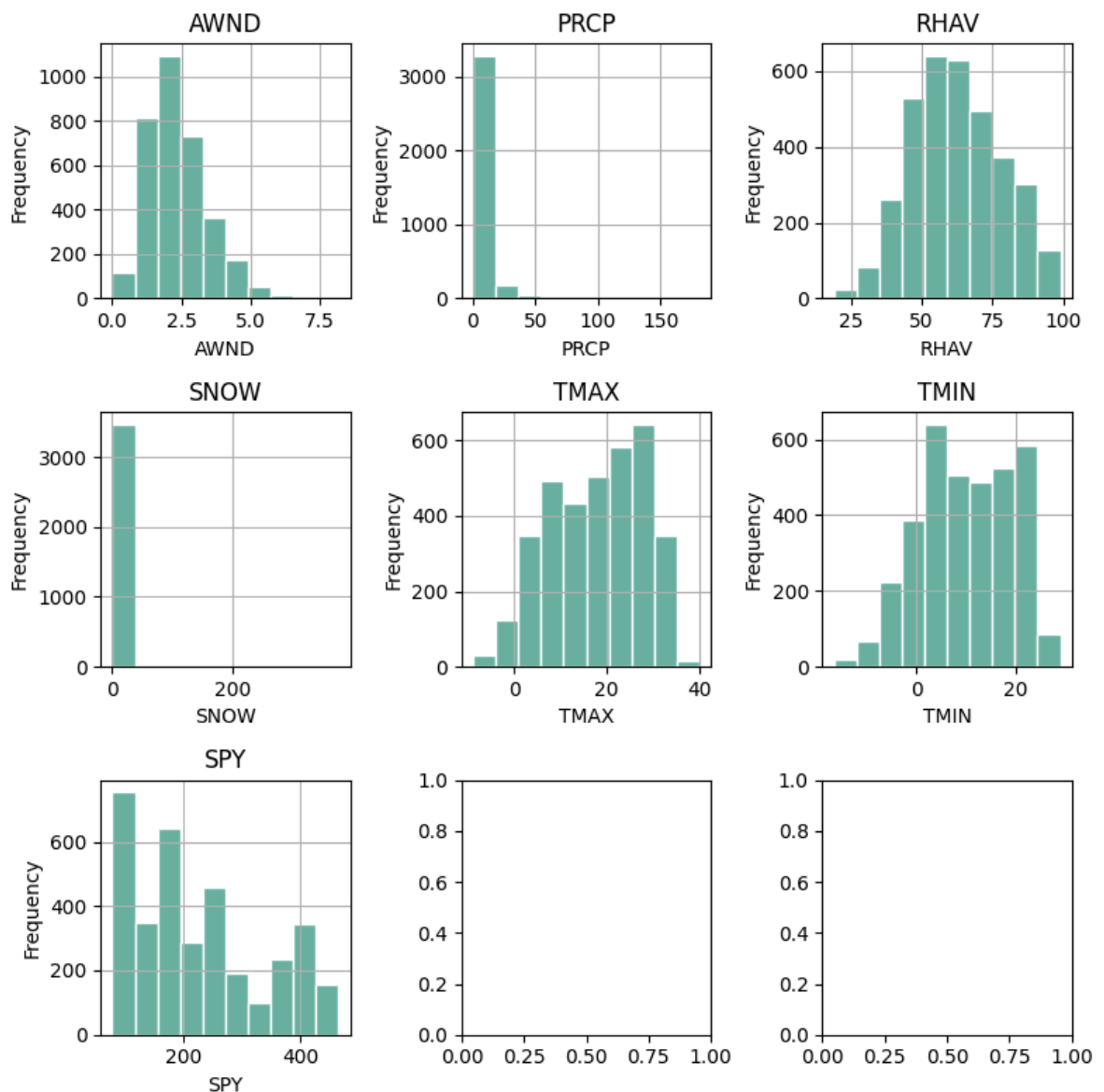
        # Add the histogram
        dataset_y[column].hist(ax=axes[i], # Define on which ax we're working o
                               edgecolor='white', # Color of the border
                               color='#69b3a2' # Color of the bins
                               )

        # Add title and axis label
        axes[i].set_title(column)
        axes[i].set_xlabel(column)
        axes[i].set_ylabel('Frequency')

    # Adjust layout
    plt.tight_layout()

    # Show the plot
    plt.show()

hist_graph(climate_stock, ["AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "SPY"], 3, 3)
```



From the above histograms, we can see there are various distributions among the variables in our dataset. Hence, issue 2 is also a problem we need to handle for the dataset. The first normalization method we are going to introduce is called standardization.

3.2 Standardization (Z-score Normalization)

Standardization (Z-Score Normalization) transforms features to have zero mean and unit variance, following the formula:

$$z = (x - \mu) / \sigma$$

where:

x is the original value

μ is the mean of the feature

σ is the standard deviation

Standardization ensures all input features contribute equally to the model, reducing bias

from features with larger scales. Standardization is suitable when the feature has a normal distribution. This method does not account for features with many outliers and non-normal distributions. It is recommended to remove outliers before conducting standardization. PCA and cluster analysis always require the dataset to be standardized first. Standardized datasets are also suitable for linear regression models and other models requiring normal distribution.

Now, let's standardize variables/features in our climate-finance dataset.

```
In [9]: # Standardize all variables in the dataset
def standardization(data_x, var_list):
    # Create a StandardScaler object
    scaler = StandardScaler()

    # Fit the scaler to the data and transform it
    data_x_2 = data_x[var_list]
    data_x_std = pd.DataFrame(scaler.fit_transform(data_x_2))
    data_x_std.columns = var_list
    return data_x_std

climate_stock_std = standardization(climate_stock, ["AWND", "PRCP", "RHAV", "SNOW", "TM
climate_stock_std.head()
```

```
Out[9]:
```

	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
0	2.299840	-0.370831	-0.372768	-0.115262	-1.901963	-1.879719	-1.271751
1	1.924686	-0.370831	-0.372768	-0.115262	-1.901963	-1.824842	-1.269712
2	1.830897	-0.370831	-0.372768	-0.115262	-1.680059	-1.451675	-1.269168
3	1.455743	-0.370831	0.069677	-0.115262	-1.508587	-1.210214	-1.265905
4	0.986800	-0.241007	0.512122	0.671979	-1.730491	-1.638258	-1.263322

```
In [10]: means = climate_stock_std.mean()
print(means)
```

```
AWND    -1.013553e-16
PRCP     4.841859e-17
RHAV     1.520537e-16
SNOW    -2.017441e-18
TMAX    -6.455811e-17
TMIN     1.634127e-16
SPY      6.455811e-17
dtype: float64
```

```
In [11]: vars = climate_stock_std.var()
print(vars)
```

```

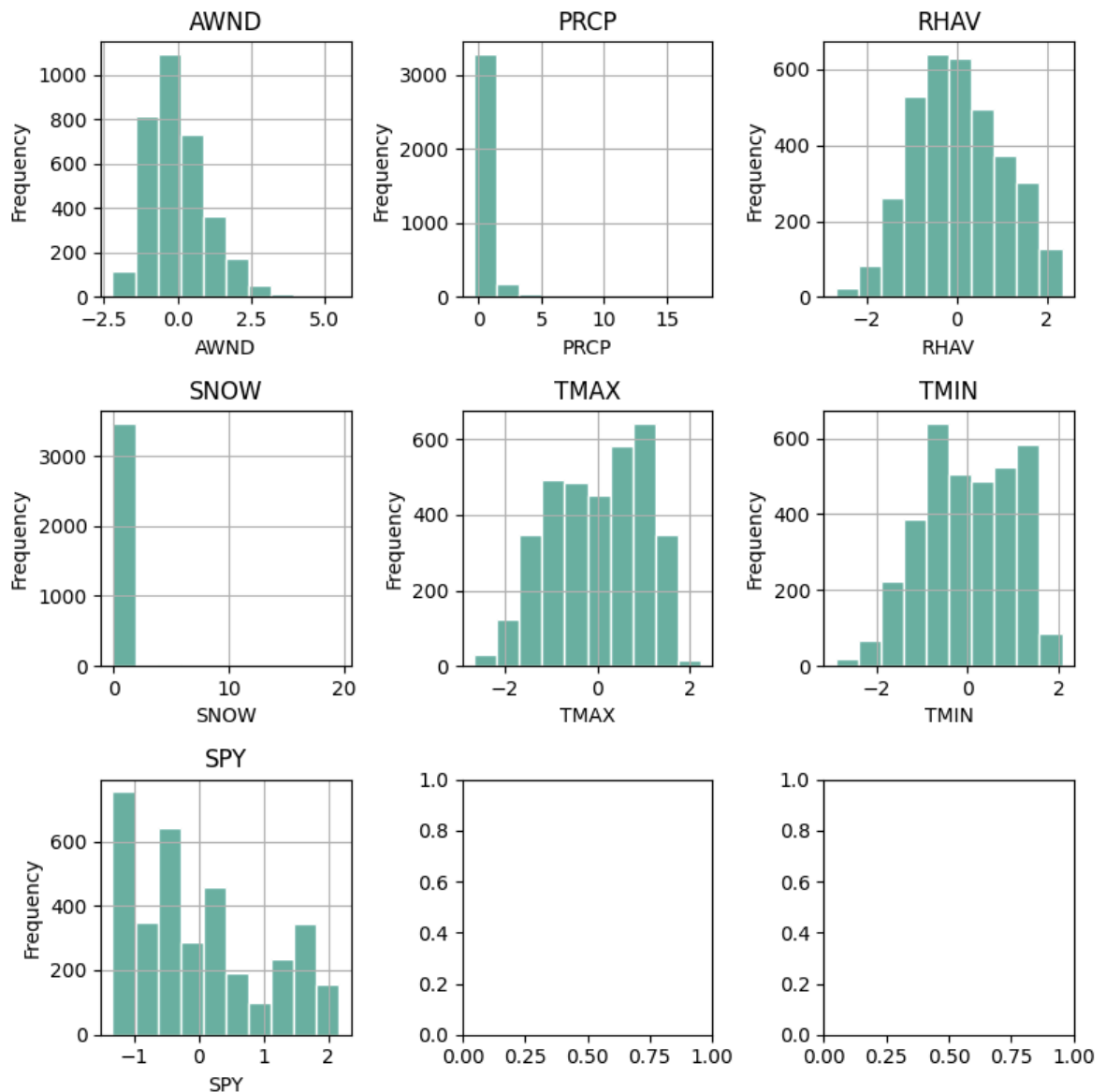
AWND    1.000297
PRCP    1.000284
RHAV    1.000289
SNOW    1.000284
TMAX    1.000284
TMIN    1.000284
SPY     1.000284
dtype: float64

```

```

In [12]: # Draw histograms of standardized variables
hist_graph(climate_stock_std, ["AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "SPY"], 3, 3

```



From the above result, we can see after standardization that the means for all standardized variables are 0 and the variances of all standardized variables are 1. Now all the standardized variables are of the same scale. The histograms of all standardized variables demonstrate the same distribution shapes as the histograms of the original variables. This standardization will preserve the shape of the data distribution.

3.3 Min-Max Scaling

Min-max Scaling, also called **normalization**, transforms features to a fixed range [0,1] using the following formula:

$$x' = (x - \min(x)) / (\max(x) - \min(x))$$

When x is $\min(x)$, then x' is 0

When x is $\max(x)$, then x' is 1

When $\min(x) < x < \max(x)$, then x' is between 0 and 1

Min-max scaling is very useful when applied to datasets for machine learning models. Datasets transformed with min-max scaling will have smaller scales and will be useful for neural network models or image processing models. Min-max scaling is suitable for a dataset that has a known upper bound and a known lower bound with few outliers. This method is also good for features without any assumptions of their distribution. Just like the standardization method, min-max scaling does not handle outliers well.

```
In [13]: # Use Min-max scaling to normalize all variables in the dataset
def min_max_scaling(data_x, var_list):
    # Create a MinMaxScaler object
    scaler_n = MinMaxScaler()

    # Fit the scaler to the data and transform it
    data_x_2 = data_x[var_list]
    data_x_nor = pd.DataFrame(scaler_n.fit_transform(data_x_2))
    data_x_nor.columns = [var_list]
    return data_x_nor

climate_stock_nor = min_max_scaling(climate_stock, ["AWND", "PRCP", "RHAV", "SNOW", "TM
climate_stock_nor.head()
```

```
Out[13]:
```

	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
0	0.585366	0.000000	0.4625	0.000000	0.157787	0.206593	0.019773
1	0.536585	0.000000	0.4625	0.000000	0.157787	0.217582	0.020353
2	0.524390	0.000000	0.4625	0.000000	0.202869	0.292308	0.020508
3	0.475610	0.000000	0.5500	0.000000	0.237705	0.340659	0.021436
4	0.414634	0.007178	0.6375	0.039894	0.192623	0.254945	0.022171

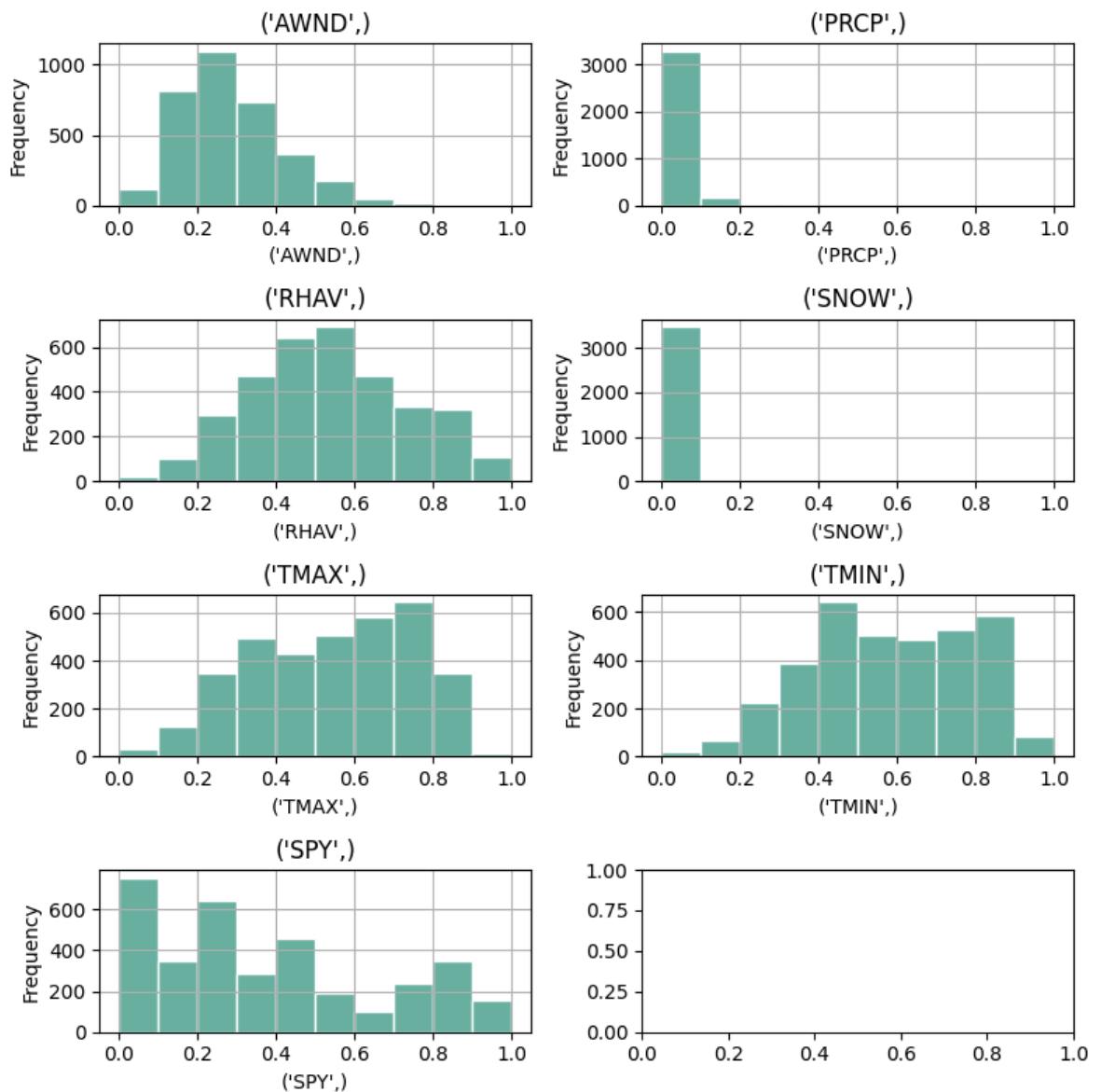
```
In [14]: # Show the minimum values of all min-max scaled/ normalized variables
mins = climate_stock_nor.min()
print(mins)
```

```
AWND    0.0
PRCP    0.0
RHAV    0.0
SNOW    0.0
TMAX    0.0
TMIN    0.0
SPY     0.0
dtype: float64
```

```
In [15]: # Show the maximum values of all min-max scaled/normalized variables
maxs = climate_stock_nor.max()
print(maxs)
```

```
AWND    1.0
PRCP    1.0
RHAV    1.0
SNOW    1.0
TMAX    1.0
TMIN    1.0
SPY     1.0
dtype: float64
```

```
In [16]: # Draw histograms of min-max scaled variables
hist_graph(climate_stock_nor, ["AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "SPY"], 4, 2)
```



The above example demonstrates how to implement min-max scaling/normalization for a dataset in Python. The example shows that after the normalization, the range of all normalized variables is between 0 and 1. The histograms of normalized variables also show the same result. Same as the standardization method, the min-max scaling method also preserves data distribution after normalization.

3.4 Robust Scaling

We learned in the previous sections that standardization and min-max scaling methods are sensitive to outliers in a dataset. In this section, we will introduce a new normalization method that can handle the outlier issue.

Robust scaling is a normalization method that is not sensitive to the existence of outliers in a dataset. Robust scaling uses a median and quartiles to avoid the extreme values in a dataset. Here is its formula.

$$x' = (x - \text{median}(x)) / \text{IQR}(x)$$

where $\text{IQR}(x)$ is the interquartile range. In this case, we use the range between the 1st quartile and 3rd quartile of x

By using statistics like median and IQR, robust scaling can avoid the use of extreme values when normalizing the dataset. Hence, robust scaling is suitable for handling datasets with outliers and also datasets of non-normal distribution.

Let's apply robust scaling to our dataset.

```
In [17]: # Use Robust scaling to normalize all variables in the dataset
def robust_scaling(data_x, var_list):
    # Create a RobustScaler object
    scaler_r = RobustScaler()

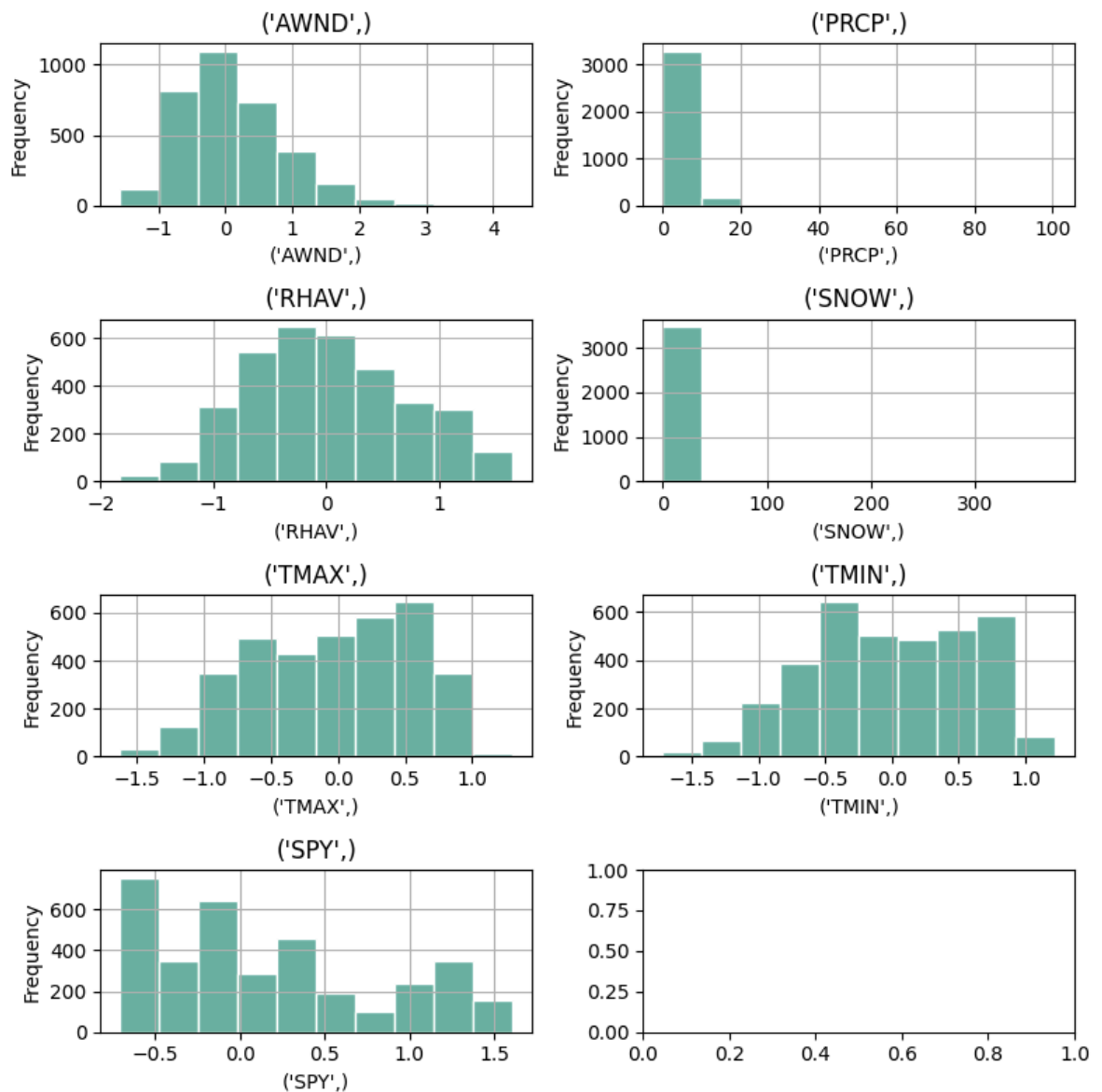
    # Fit the scaler to the data and transform it
    data_x_2 = data_x[var_list]
    data_x_rob = pd.DataFrame(scaler_r.fit_transform(data_x_2))
    data_x_rob.columns = [var_list]
    return data_x_rob

climate_stock_rob = robust_scaling(climate_stock, ["AWND", "PRCP", "RHAV", "SNOW", "TMA
climate_stock_rob.head()
```

```
Out[17]:
```

	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
0	1.857143	0.000000	-0.217391	0.0	-1.161677	-1.109677	-0.658602
1	1.571429	0.000000	-0.217391	0.0	-1.161677	-1.077419	-0.657263
2	1.500000	0.000000	-0.217391	0.0	-1.029940	-0.858065	-0.656905
3	1.214286	0.000000	0.086957	0.0	-0.928144	-0.716129	-0.654762
4	0.857143	0.722222	0.391304	15.0	-1.059880	-0.967742	-0.653065

```
In [18]: # Draw histograms of robust scaled variables
hist_graph(climate_stock_rob, ["AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "SPY"], 4, 2
```



We can see from the above histograms that robust scaling also maintains the original data distribution shape.

3.5 Time Series Temporal Normalization

So far, we have been discussing various normalization methods for a dataset in general. In this section, we will focus on time series data. We will also introduce a method for time series temporal normalization.

In machine learning, we usually talk about feature engineering. **Feature engineering** is a technique to transform raw data for the use of machine learning models. Normalization is a method of feature engineering.

Time series data is special in the sense that it has the additional temporal feature in the dataset. For time series normalization, we can use the above discussed normalization methods. However, those methods do not account for the time element in the time series

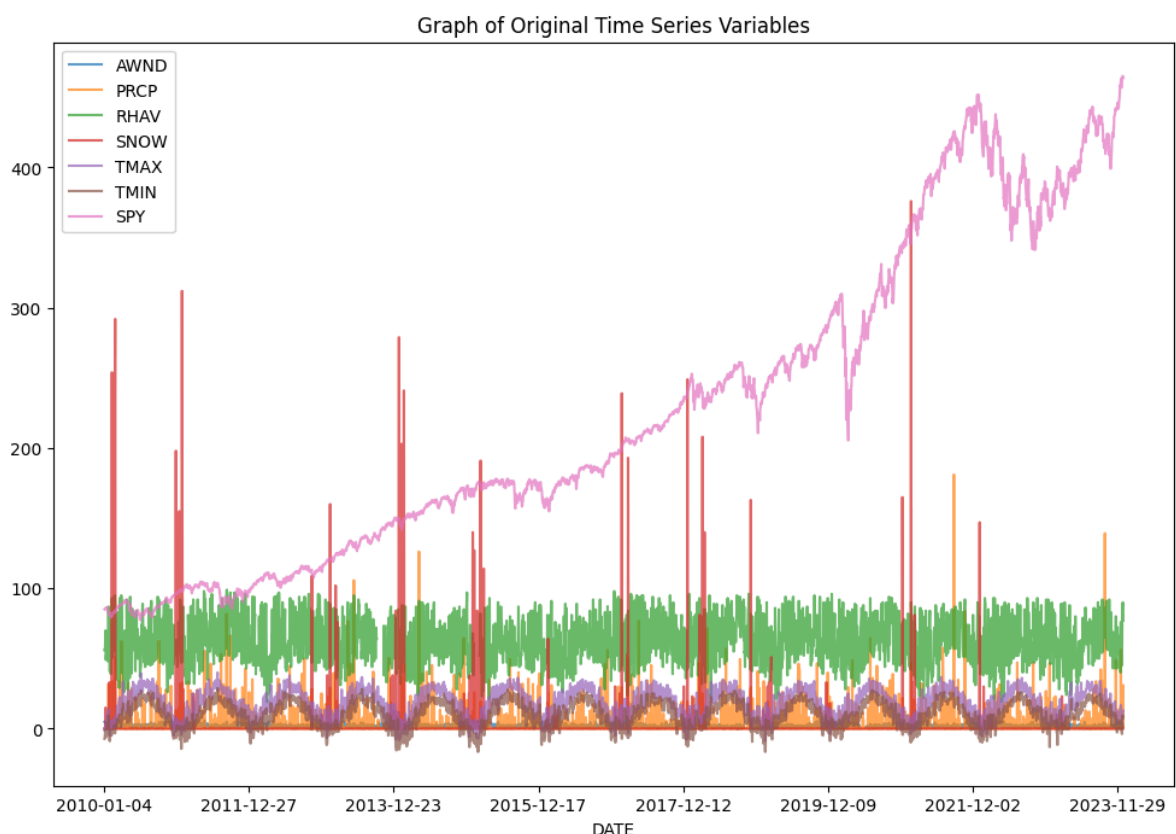
data.

In this section, we are going to introduce a **rolling average** normalization method for time series data. Different from the standardization method using mean and standard deviation of the whole dataset, rolling average normalization uses the most recent data points to calculate the mean and the standard deviation. Hence, a user needs to decide how far back the data should be used to calculate the mean and the standard deviation. We call this looking back period "Window." For example, if we would like to use data from the latest 30 days to calculate mean and standard deviation, the window is 30 days. We call this mean and standard deviation the rolling mean and rolling standard deviation. We then apply the rolling mean and rolling standard deviation to each data point as we use the standardization method.

Instead of using a static mean and standard deviation, rolling average normalization uses a dynamic mean and standard deviation to standardize time series data.

Let's first look at the time series graph for all variables in our dataset.

```
In [19]: # Original Time Series graph
climate_stock_g = climate_stock[["DATE", "AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "
climate_stock_g.set_index('DATE', inplace=True)
climate_stock_g.plot(figsize=(12, 8), title='Graph of Original Time Series Variable
plt.show()
```



From the above graph, SPY has a significant trend. The rest of the climate variables all have a clear seasonal pattern. Now let's implement the rolling average normalization in Python.

```
In [20]: # Rolling Average normalization for time series data
def normalize_temporal_features(X, window_size):
    rolling_mean = X.rolling(window=window_size).mean()
    rolling_std = X.rolling(window=window_size).std()
    return (X - rolling_mean) / rolling_std

climate_stock_t_n = normalize_temporal_features(climate_stock[["AWND", "PRCP", "RHAV",
climate_stock_t_n.columns = ["AWND", "PRCP", "RHAV", "SNOW", "TMAX", "TMIN", "SPY"]
climate_stock_t_n.head(50)
```

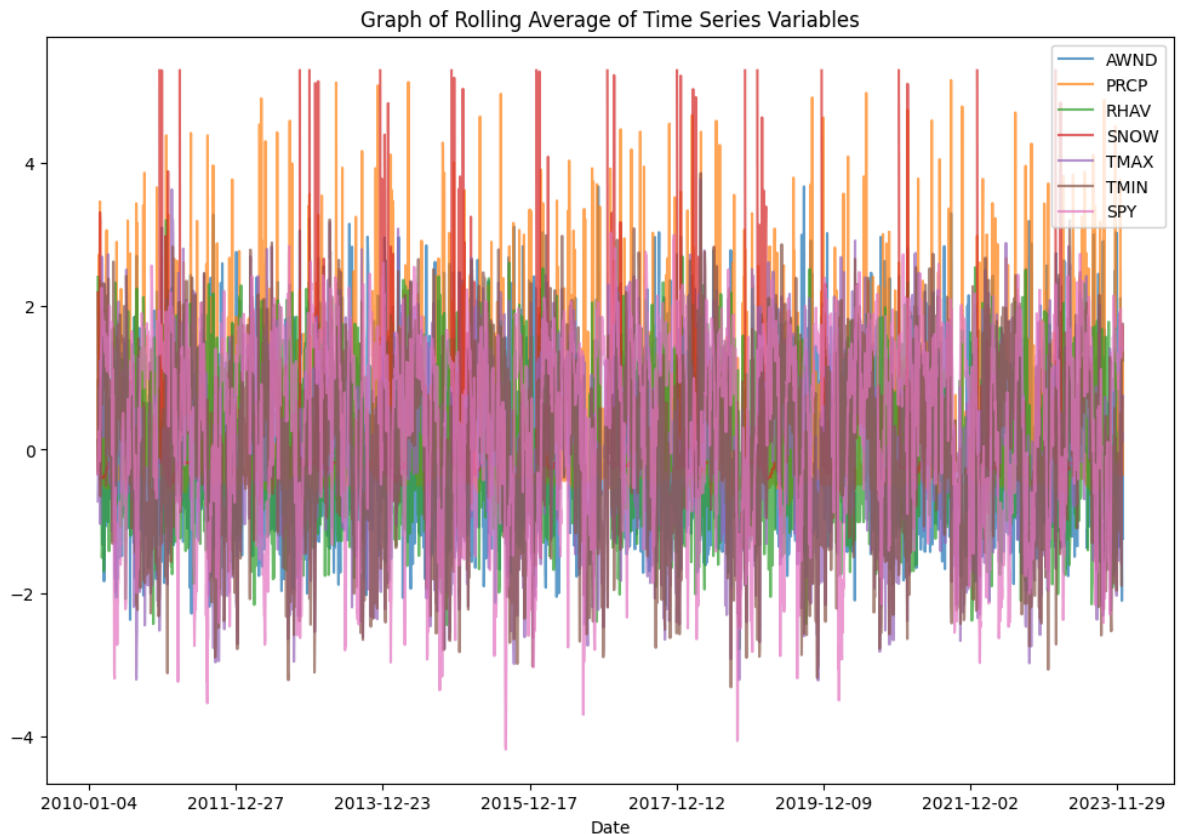
Out[20]:

	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	NaN	NaN	NaN	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN
12	NaN	NaN	NaN	NaN	NaN	NaN	NaN
13	NaN	NaN	NaN	NaN	NaN	NaN	NaN
14	NaN	NaN	NaN	NaN	NaN	NaN	NaN
15	NaN	NaN	NaN	NaN	NaN	NaN	NaN
16	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	NaN	NaN	NaN	NaN	NaN	NaN	NaN
19	NaN	NaN	NaN	NaN	NaN	NaN	NaN
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN
21	NaN	NaN	NaN	NaN	NaN	NaN	NaN
22	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN	NaN	NaN	NaN	NaN	NaN	NaN
24	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25	NaN	NaN	NaN	NaN	NaN	NaN	NaN
26	NaN	NaN	NaN	NaN	NaN	NaN	NaN
27	NaN	NaN	NaN	NaN	NaN	NaN	NaN
28	NaN	NaN	NaN	NaN	NaN	NaN	NaN
29	NaN	0.968320	2.409570	2.192237	-0.723449	0.132641	-0.348141

	AWND	PRCP	RHAV	SNOW	TMAX	TMIN	SPY
30	NaN	-0.326204	0.650828	-0.296566	-0.469979	-0.349985	-0.146360
31	NaN	-0.326204	0.119266	-0.296566	0.207461	0.529873	0.100405
32	NaN	-0.326204	-0.250817	-0.296566	0.628901	0.959993	0.210584
33	NaN	-0.234576	-0.014932	-0.296566	0.879546	-0.003804	0.259208
34	NaN	2.727351	2.369750	-0.286348	-0.205391	0.903708	-0.167348
35	NaN	0.668848	1.835628	-0.286348	0.380740	0.981952	0.263437
36	NaN	3.462625	1.982958	3.309942	-0.286874	-0.488808	0.252198
37	NaN	1.678064	1.548452	3.181511	-1.035887	-0.652049	0.352389
38	NaN	-0.497023	-0.237637	-0.396157	1.243264	0.928442	0.947120
39	NaN	-0.475960	0.135491	-0.396157	1.117864	0.919653	1.147318
40	NaN	-0.371196	0.806747	-0.396157	0.233537	0.656628	1.355644
41	NaN	-0.503330	-0.611324	-0.396157	1.131443	0.827500	1.620465
42	NaN	-0.503330	-1.186381	-0.396157	1.355564	0.626402	2.257567
43	NaN	-0.503330	-1.508524	-0.396157	2.691720	1.653896	2.020610
44	-1.317841	-0.451845	-1.407548	-0.396157	2.564765	2.321690	1.900487
45	-1.082241	-0.451845	-0.900482	-0.396157	1.927819	2.107473	1.905207
46	0.361591	-0.451845	0.458374	-0.396157	1.350895	1.651652	1.890221
47	1.945772	-0.046570	1.086420	-0.381685	0.175063	1.131418	1.724025
48	1.186693	0.174095	0.967871	-0.381685	0.481010	1.413329	1.600570
49	-0.174496	-0.483656	-1.698575	-0.381685	2.292993	1.319830	1.746707

Let's draw a time series graph for the normalized data to better understand the change after applying rolling average normalization.

```
In [21]: # Rolling Average Time series graph
climate_stock_t_n["Date"] = climate_stock["DATE"]
climate_stock_t_n.set_index('Date', inplace=True)
climate_stock_t_n.plot(figsize=(12, 8), title='Graph of Rolling Average of Time Ser
plt.show()
```



From the above graph, we can see that normalized SPY just oscillating around 0 without any discernible trend. Other climate variables also oscillate around 0 but still demonstrate a seasonal pattern. With this exercise, we can easily detect seasonal patterns and trends for temporal climate data.

3.6 Integration for Multi-Normalization Features

We introduced several normalization methods in the previous sections. Sometimes we would like to apply different normalization methods to different features. We can normalize a feature with a specific normalization method and then put all normalized features into one dataset for analysis. Let's demonstrate this with the following Python code.

```
In [22]: #Normalize variables based on the following rules
#Standardization : AWND, TMAX, TMIN
# Min-max Scaling: RHAV
# Robust Scaling: PRCP, SNOW
# Rolling Average: SPY

combined_data = pd.DataFrame(climate_stock["DATE"])
combined_data[["AWND", "TMAX", "TMIN"]] = standardization(climate_stock, ["AWND", "TMAX", "TMIN"])
combined_data[["RHAV"]] = min_max_scaling(climate_stock, ["RHAV"])
combined_data[["PRCP", "SNOW"]] = robust_scaling(climate_stock, ["PRCP", "SNOW"])
combined_data[["SPY"]] = normalize_temporal_features(climate_stock[["SPY"]], 30)
combined_data.set_index('DATE', inplace=True)
```

```
combined_data.head(50)
```

Out[22]:

	AWND	TMAX	TMIN	RHAV	PRCP	SNOW	SPY
DATE							
2010-01-04	2.299840	-1.901963	-1.879719	0.4625	0.000000	0.0	NaN
2010-01-05	1.924686	-1.901963	-1.824842	0.4625	0.000000	0.0	NaN
2010-01-06	1.830897	-1.680059	-1.451675	0.4625	0.000000	0.0	NaN
2010-01-07	1.455743	-1.508587	-1.210214	0.5500	0.000000	0.0	NaN
2010-01-08	0.986800	-1.730491	-1.638258	0.6375	0.722222	15.0	NaN
2010-01-11	0.799223	-1.851530	-1.758989	0.3750	0.000000	0.0	NaN
2010-01-12	1.737109	-1.619539	-1.572406	0.4250	0.000000	0.0	NaN
2010-01-13	1.174377	-1.619539	-1.572406	0.4250	0.000000	0.0	NaN
2010-01-14	-0.795182	-1.226164	-1.210214	0.4375	0.000000	0.0	NaN
2010-01-15	1.080589	-0.782355	-0.902901	0.3875	0.000000	0.0	NaN
2010-01-19	NaN	-0.893307	-0.606562	0.6250	0.000000	0.0	NaN
2010-01-20	0.424069	-1.286683	-0.968754	0.5250	0.000000	0.0	NaN
2010-01-21	-0.888970	-1.286683	-1.155337	0.4625	0.000000	0.0	NaN
2010-01-22	0.517858	-1.115212	-1.089484	0.3625	0.000000	0.0	NaN
2010-01-25	NaN	-0.388980	-0.057788	0.8500	17.666667	0.0	NaN
2010-01-26	1.455743	-0.842875	-0.902901	0.4000	0.000000	0.0	NaN
2010-01-27	0.705435	-1.397635	-1.155337	0.4125	0.000000	0.0	NaN
2010-01-28	1.361954	-1.226164	-1.824842	0.4875	1.277778	33.0	NaN
2010-01-29	2.674994	-2.295338	-2.066303	0.2750	0.000000	0.0	NaN
2010-02-01	1.174377	-1.619539	-1.517528	0.2875	0.000000	0.0	NaN
2010-02-02	-0.420028	-1.730491	-1.396798	0.4375	0.444444	20.0	NaN
2010-02-03	-0.044874	-1.397635	-1.276067	0.6750	0.833333	5.0	NaN
2010-02-04	1.549532	-1.508587	-1.330945	0.2500	0.000000	0.0	NaN
2010-02-05	-0.232451	-1.569106	-1.276067	0.3750	0.000000	0.0	NaN
2010-02-08	1.924686	-1.508587	-1.638258	0.3875	0.000000	0.0	NaN
2010-02-09	-0.044874	-1.226164	-1.330945	0.4625	0.000000	0.0	NaN
2010-02-10	2.956359	-1.680059	-1.210214	0.9250	18.777778	254.0	NaN
2010-02-11	2.768782	-1.508587	-1.330945	0.4750	0.000000	0.0	NaN
2010-02-12	1.268166	-1.508587	-1.451675	0.4000	0.000000	0.0	NaN

	AWND	TMAX	TMIN	RHAV	PRCP	SNOW	SPY
DATE							
2010-02-16	0.142704	-1.730491	-1.276067	0.8875	6.055556	127.0	-0.348141
2010-02-17	1.174377	-1.619539	-1.451675	0.6000	0.000000	0.0	-0.146360
2010-02-18	2.018474	-1.347202	-1.089484	0.5125	0.000000	0.0	0.100405
2010-02-19	2.581205	-1.175731	-0.902901	0.4500	0.000000	0.0	0.210584
2010-02-22	0.330281	-1.064779	-1.276067	0.4875	0.444444	0.0	0.259208
2010-02-23	3.706668	-1.458154	-0.902901	0.9375	16.944444	0.0	-0.167348
2010-02-24	1.080589	-1.226164	-0.848023	0.8750	5.944444	0.0	0.263437
2010-02-25	-0.795182	-1.458154	-1.396798	0.9500	28.222222	239.0	0.252198
2010-02-26	0.705435	-1.730491	-1.451675	0.8875	16.500000	292.0	0.352389
2010-03-01	1.924686	-0.893307	-0.848023	0.5000	0.000000	0.0	0.947120
2010-03-02	0.048915	-0.953827	-0.848023	0.5875	0.166667	0.0	1.147318
2010-03-03	2.206051	-1.286683	-0.968754	0.7375	1.000000	0.0	1.355644
2010-03-04	1.080589	-0.953827	-0.902901	0.4250	0.000000	0.0	1.620465
2010-03-05	0.986800	-0.842875	-0.968754	0.2875	0.000000	0.0	2.257567
2010-03-08	0.799223	-0.167075	-0.540710	0.2000	0.000000	0.0	2.020610
2010-03-09	-0.326239	-0.167075	-0.354126	0.2000	0.000000	0.0	1.900487
2010-03-10	-0.138662	-0.388980	-0.354126	0.3125	0.000000	0.0	1.905207
2010-03-11	1.549532	-0.610884	-0.474857	0.6375	0.000000	0.0	1.890221
2010-03-12	3.612879	-1.175731	-0.661440	0.8000	2.944444	0.0	1.724025
2010-03-15	2.674994	-1.004259	-0.540710	0.7875	4.666667	0.0	1.600570
2010-03-16	0.986800	-0.056123	-0.540710	0.1375	0.000000	0.0	1.746707

4. Sparse Climate Data

Sparse climate data represents one of the most significant challenges in climate science. When scientists attempt to analyze climate patterns, they often encounter missing values in data that can span years, decades, or even centuries. These missing values can be due to disruptive events or natural disasters that prevented the collection of data. It can also be the inconsistent frequency of data collection. Remote locations or other geographical restrictions can also make data inaccessible. Sometimes, the equipment or sensor issues can also disrupt the collection of data.

4.1 Challenges and Mitigation Measures for Sparse Climate Data

With the data gap, there will be challenges to scientific research and statistical analysis. Without complete data, the implications for scientific research are low confidence for trend analysis, signals that are hard to identify, difficulty with model validation, and low accuracy for weather forecasts. For statistical analysis, the uncertainty of the prediction will be wider and the statistical power will be lower.

In order to mitigate the impact of sparse climate data, scientists have a few measures they can take. The first one is to use imputation techniques to add imputed values to the dataset. The other one is the interpolation technique. We won't go into details for these methods.

4.2 Compression of Sparse Data Matrix

In the pervious section, we talked about the challenges using a sparse matrix for analysis and methods to mitigate the issue. In this section, we are going to discuss another issue with sparse matrix: data storage. As described previously, a sparse matrix contains a lot of zeros in the matrix. To store the sparse matrix with all the zeros in the memory will waste a lot of computer space. In machine learning, it's essential to break up the data matrix in the memory so that it's an appropriate size and move it to the RAM before computing. Using the original sparse matrix for modeling will take up a lot of resources and reduce the computer's efficiency. For this reason, there are several sparse matrix compression methods that have been developed. One of the most popular methods is the compressed sparse row.

4.2.1 Compressed Sparse Row Format

Compressed sparse row or CSR is the method of compressing data from a sparse data matrix to reduce data storage space. In general, a sparse row is a row where half of its values are zeros. Let's use the following example to explain what CSR is and how it works.

Suppose we have the following matrix:

$$\begin{bmatrix} 3 & 0 & 0 & 7 & 0 & 2 & 10 & 9 & 6 & 0 & 5 & 0 & 0 & 8 & 13 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 7 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 15 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

In order to convert this matrix into a CSR format, we first need to count how many non-zero numbers we have in the matrix. In the above matrix, there are 12 non-zero values. Let's give all these non-zero values an id and present them in the following table.

Table 1

Non-Zero ID	0	1	2	3	4	5	6	7	8	9	10	11
Value	3	7	2	10	9	6	5	8	13	4	7	15

In the above table 1, we start from the top row and go from left to right to record each non-zero number and assign an id to it. The next step is to add column id to table 1. We will assign column id 0 to the values on the left-most column and column id 4 to the values on the right-most column. Column id 1, 2, 3 will be assigned to the values on the columns in between. The result will be as follows.

Table 2

Non-Zero ID	0	1	2	3	4	5	6	7	8	9	10	11
Value	3	7	2	10	9	6	5	8	13	4	7	15
Column ID	0	3	0	1	2	3	0	3	4	2	3	4

The next information we will need is row pointer. To find this information, first, we need to identify the first non-zero number (from left to right) of each row. In our matrix, the first non-zero numbers of rows are 3, 2, 5, 4, and 15 as highlighted in yellow in the following matrix.

$$\begin{bmatrix} \textcolor{yellow}{3} & 0 & 0 & 7 & 0 & \textcolor{yellow}{2} & 10 & 9 & 6 & 0 & \textcolor{yellow}{5} & 0 & 0 & 8 & 13 & 0 & 0 & \textcolor{yellow}{4} & 7 & 0 & 0 & 0 & 0 & 0 & \textcolor{yellow}{15} \end{bmatrix}$$

Once we identify the first non-zero numbers for all rows, we can mark them with their corresponding Non-Zero ID on the table 2 as follows:

Table 3

Non-Zero ID	$\textcolor{yellow}{\boxed{0}}$	1	$\textcolor{yellow}{\boxed{2}}$	3	4	5	$\textcolor{yellow}{\boxed{6}}$	7	8	$\textcolor{yellow}{\boxed{9}}$
Value	$\textcolor{yellow}{\boxed{3}}$	7	$\textcolor{yellow}{\boxed{2}}$	10	9	6	$\textcolor{yellow}{\boxed{5}}$	8	13	$\textcolor{yellow}{\boxed{4}}$
Column ID	0	3	0	1	2	3	0	3	4	2

Then, we can collect all highlighted Non-Zero IDs from left to right in table 3 and put them in an array as follows:

$\left[0, 2, 6, 9, 11 \right]$

In order to provide ending information to the CRS, we need to add one extra number to the right side of this array. The number to add to the array is **the largest Non-Zero ID + 1**. In our case it is $11 + 1 = 12$. Let's add 12 to the right side of this array.

$\left[0, 2, 6, 9, 11, 12 \right]$

The above array is our row pointer array.

Now we can assemble all the information we collected to form CSR. For the CSR format of our matrix, there are 3 one-dimensional arrays.

Value array: $\left[3, 7, 2, 10, 9, 6, 5, 8, 13, 4, 7, 15 \right]$

Column array: $\left[0, 3, 0, 1, 2, 3, 0, 3, 4, 2, 3, 4 \right]$

Row pointer array: $\left[0, 2, 6, 9, 11, 12 \right]$

With only 3 one-dimensional arrays in CSR format, we can store the information of a sparse matrix with many zeros in a more efficient way. This compression technique not only saves storage space and utilizes computer memory more efficiently but also optimizes the processing of many machine learning models.

Now we know the basics of CSR. Let's use Python to derive a CSR format for our matrix.

```
In [23]: # Set up the matrix
s_matrix = np.matrix([[3,0,0,7,0],
                      [2,10,9,6,0],
                      [5,0,0,8,13],
                      [0,0,4,7,0],
                      [0,0,0,0,15]])

# Convert to CSR format
csr = csr_matrix(s_matrix)

# Print value array
csr_value = csr.data
csr_value
```

```
Out[23]: array([ 3,  7,  2, 10,  9,  6,  5,  8, 13,  4,  7, 15])
```

```
In [24]: # Print column id array
csr_col = csr.indices
csr_col
```

```
Out[24]: array([0, 3, 0, 1, 2, 3, 0, 3, 4, 2, 3, 4], dtype=int32)
```

```
In [25]: # Print row pointer array
csr_row_ptr = csr.indptr
csr_row_ptr
```

```
Out[25]: array([ 0,  2,  6,  9, 11, 12], dtype=int32)
```

From the above Python result, we can use Python to produce three arrays of CSR format for our matrix. Now let's try to do the reverse. Suppose I have the CSR format information about a sparse matrix, let's use Python to reconstruct the matrix.

```
In [26]: # Reconstruct the matrix from 3 arrays from CSR
mtx = csr_matrix((csr_value, csr_col, csr_row_ptr)).toarray()
mtx
```

```
Out[26]: array([[ 3,  0,  0,  7,  0],
                [ 2, 10,  9,  6,  0],
                [ 5,  0,  0,  8, 13],
                [ 0,  0,  4,  7,  0],
                [ 0,  0,  0,  0, 15]])
```

The above `csr_matrix` function uses CSR format as input to reconstruct the matrix. Now we learn how to compress a sparse matrix to a CSR format and reconstruct a matrix using CSR format as input. There are several other compression methods to compress a sparse matrix. We will not dive in more. For readers who are interested in this topic, there are plenty of resources available online.

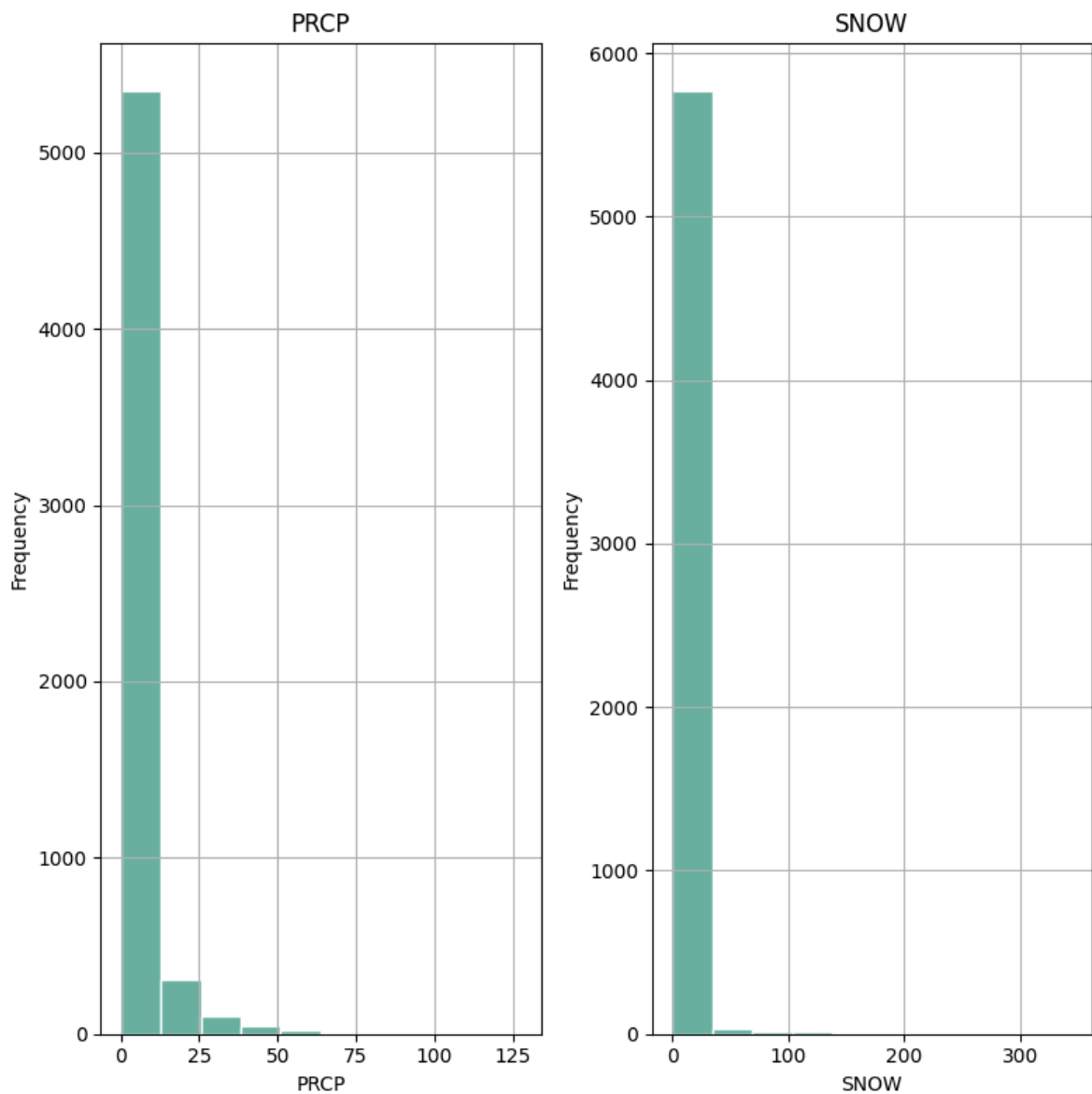
4.2.2 CSR Application

In this section, we will illustrate how to convert a sparse climate data matrix into CSR format. First let's pull some climate data. In this case, we are only going to pull precipitation and snowfall data from 1985 to 2000.

```
In [27]: # Pull precipitation data and snowfall data from 1985 to 2000 in NYC
#Set parameters
dataset = "daily-summaries"           # this is the name for GHCN-Daily dat
vars = "PRCP, SNOW"                   # climate variable symbols
station_id = "USW00094728"            # New York Central Park station ident
st_dt = "1985-01-01"                  # start date
end_dt = "2000-12-31"                 # end date

api_base = "https://www.ncei.noaa.gov/access/services/data/v1?"
req_params = "dataset=" + dataset + "&dataTypes=" + vars + "&stations=" + station_id
req_str = api_base + req_params
data_2 = requests.get(req_str)
```

```
In [28]: #Convert the downloaded data into a pandas dataframe
s_2=str(data_2.content,'utf-8')
data_s_2 = StringIO(s_2)
climate_2=pd.read_csv(data_s_2)
hist_graph(climate_2, ["PRCP", "SNOW"],1,2)
```



From the above histograms, we see that there are a lot of zeros for these two climate variables in the dataset. Let's convert this data into a matrix and then convert to CSR format.

```
In [29]: # Pull only data for CSR conversion
climate_2_df = climate_2[["PRCP", "SNOW"]]

# Convert to CSR format
climate_csr = csr_matrix(climate_2_df)
```

```
In [30]: # Show the value array
climate_csr.data
```

```
Out[30]: array([ 0.8,  5.1,  2. , ..., 20. , 23.4, 305. ])
```

```
In [31]: # Show the column id array
climate_csr.indices
```

```
Out[31]: array([0, 0, 0, ..., 1, 0, 1], dtype=int32)
```

```
In [32]: # Show row pointer array  
climate_csr.indptr
```

```
Out[32]: array([ 0, 1, 2, ..., 2113, 2115, 2115], dtype=int32)
```

5. Conclusion

In this lesson, we first learned why a data matrix needs to be normalized. Then, we introduced 4 methods to normalize a data matrix: standardization, min-max scaling, robust scaling, and time series temporal normalization. We then discussed the challenges and mitigation methods for a sparse data matrix for analysis. Lastly, we talked about how inefficiently a sparse data matrix can be in terms of storage space and computation process. We introduced a method to compress a sparse matrix: compressed sparse row format. CSR reduces the storage requirement of a sparse matrix and makes model calculation with a sparse matrix more efficient.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Lesson 3 - Numerical Methods for Data Preparation and Statistical Analysis

December 25, 2025

MODULE 7 | LESSON 3

1 Numerical Methods for Data Preparation and Statistical Analysis

Reading Time	45 minutes
Prior Knowledge	Basic understanding of quantitative areas: linear algebra, calculus, statistical inference, and regression
Keywords	Imputation, Calibration, Feature Extraction

1.1 1. Data Cleaning

In this first section, we review two groups of numerical methods: those for imputing and those for excluding.

1.1.1 1.2 Imputing: Filling in Missing Values

When we are given data, we may notice that some of the values are missing. Imputation is the name given to the action of providing missing values. We will learn various methods to fill in missing data, some of which are quite simple while others will instead involve a fair amount of complexity.

On the simple side, we have methods that simply use the last known value (e.g., fill forward). We could also replace a missing value by looking at its neighbors and performing some degree of interpolation: linear, quadratic, cubic, quartic, splines, etc. We'll also use resampling methods to handle missing or censored data. As we build our stochastic modeling skills, we'll look at the expectation maximization algorithm and the Brownian bridge approach. One data-imputing application we learned in this course (Module 1, Lesson 4) is to use a cubic spline equation to fit a yield curve. In the section Cubic Spline Fitting of Yield Curve, we use a cubic spline equation to estimate missing yield information from the yield dataset. With this method, we are able to fit a smooth and piecewise yield curve for the following yield curve analysis.

1.1.2 1.2 Excluding: Outlier Detection

The previous subsection discussed how to handle missing values. We will also develop skills in the opposite direction: taking existing data values and excluding them, a process known as outlier detection. We will learn various methods for labeling and discarding outliers. Outliers are observations that cause the model's results to be extremely sensitive to those points. To understand outliers, we'll need to use both exploratory data analysis (prior to modeling) and diagnostic plots (post modeling). Exploratory data analysis is early visualization of raw data. Diagnostic plots show the sensitivity of data points to model parameter values and estimation.

Under the same umbrella of excluding, we'll look at dropping not just rows but also entire variables. Indeed, some of the methods we'll use take care of this automatically. For example, the Lasso method drops variables based on a parameter. Discarding the most egregious outliers could result in getting different parameters, which we can interpret and apply very differently. However, in our taxonomy, we'll consider feature extraction a separate category. In the next course, Financial Econometrics, we will dive deeper into the impact of outliers on regression models, formal methods to detect them, and specialized regression methods to handle outliers.

1.2 2. Data Engineering

1.2.1 2.1 Transforming and Normalizing

Sometimes, only a simple transformation is needed. A transform could be as simple as a conversion (e.g., degrees Fahrenheit to degrees Celsius), where the conversion factor never changes. Other times, the conversion factor does change: for example, converting Japanese yen to Nigerian naira.

What are some of the transforms we'll do? You've already transformed financial data series using logs. You may transform prices into volume-weighted average prices or VWAPs. We'll do this in the next lesson. You can transform prices to returns. You can transform financial time series using lags so that a series can be regressed on itself or lagged versions of other series. Some transformations are more complex, like a Fourier transformation, which will prove to be useful in option pricing.

Sometimes, we need to transform data to make it more usable through better-behaved measures such as having a zero-mean or a unit variance. Another set of transformations, called normalization, can have multiple meanings that depend on the context. For example, this includes the transformations that relate to making data more Gaussian: symmetric and bell shaped. In other cases, it is used interchangeably with standardization, where we subtract the mean and divide by the standard deviation. Lastly, it is used when we want to eliminate the effects of external factors, which is what we do when we detrend a time series. The purpose of these normalizations is to facilitate the analysis of the features of a dataset. Almost every example we'll do has one type of transformation. Over time, these will become second nature.

In this course, you have already transformed daily closing prices into logarithmic or percent returns. You have also seen that daily closing prices themselves are a transformation of raw tick-by-tick data. Furthermore, we computed the adjusted-close price by transforming the closing prices using data for splits and dividends. So, one can see that even the financial data readily available from multiple vendors are already heavily cleaned and transformed. In the rest of the Financial Engineering course, you will dive deep into multiple transformations such as standardization for comparing different assets on the same scale, Fourier transformations for option pricing, and many more. You'll learn not just how to apply these transformations but also understand their mathematical foundations and, most importantly, when and why to use them in real-world scenarios.

1.2.2 2.2 Factoring and Extracting

Factoring and feature extraction are another way of reorganizing data. Factoring is a difficult process because it involves both identification of factors and their measurement. There are a bewildering number of variations of forming factors. Principal components can be a first step, but these components are often not very intuitive. So sometimes, we simplify the problem by using classical features engineering to shape data that can be used in many ML algorithms. Less straightforward methods require a serious amount of subject matter expertise to define relevant factors. Within the professional world, you will encounter factor models, such as the FICO scores in credit risk. On the portfolio management and risk management sides, you will encounter commercially available factor models like the MSCI Barra Factor Index. There are different versions for different regions of the world.

Incidentally, the British refer to factor modeling as latent variable analysis: a good name to indicate that the variables need to be extracted. An example of a latent variable comes from the book (and Hollywood movie) *Moneyball*. For many years, it was thought that the most important metric of a baseball player was the number of hits or homeruns. However, a less obvious but more valuable factor is the player's percentage of getting on base. Aside from getting hits, there are two other ways to get on base. One way is to walk. The other is to get struck by a pitch. Whether the batter gets a hit, a walk, or hit by a pitch, the batter winds up on base and can potentially score, which is what wins games. The percent-on-base metric proved to be more important for winning games than the more easily observed number of hits. The “percent-on-base” factor is an example of a latent variable. One has to form a ratio: total times the batter gets on base from any of the three methods divided by the total number of times the batter has an official plate appearance.

One of the complexities of finding factors is overcoming biases. After writing *Moneyball*, Michael Lewis wrote *The Undoing Project*. In that book, he explains why walks were not considered as valuable as hits. In order for a batter to draw a walk, he must avoid swinging his bat four times when the pitches are not in the strike zone. From one perspective, it appears that the batter is taking no action. On the other hand, when the batter swings and connects the ball, there is the crack of the bat, running by both the batter and the players in the field, and applause by the fans—all reactions that give the illusion that the action is more valuable than the motionless stance of a walk, which typically draws less applause. However, both a single and a walk have the same outcome: the batter proceeds to first base. The cognitive bias is one of discounting a result because it was achieved by little action—not swinging. But the batter is exhibiting excellent skill with a good eye on a fast-moving ball and restraint from chasing bad pitches.

Think back to Module 1. You've already witnessed the power of creating factors when you “factored out” the level and slope of the yield curve using the first and second principal components of the interest rates. In Financial Engineering, you'll dive deeper into advanced factoring techniques like factor analysis (FA), which goes beyond PCA by attempting to explain the correlations among variables through a smaller set of **interpretable** factors. These techniques are not just academic exercises; they're used by industry leaders to build advanced factor models for portfolio management and risk assessment.

Being able to extract features and define factors is one of the most valuable skills you can build as a financial engineer.

1.3 3. Statistical Analysis

1.3.1 3.1 Inferencing and Estimating: Parameter Evaluation

Flush with data, we will be well equipped to work on inference problems. Given samples of data, how do we best estimate the parameters of the population? What is a population's mean? Median? Standard deviation? First percentile? Skewness? Kurtosis? Correlation to another variable?

Suppose you assume the yield curve follows a polynomial model given by Nelson Siegel. Then, you can estimate the NS parameters from the data. This exercise is one you performed back in Module 1. Thanks to advances in computational power and contributions from fields outside statistics, there are many specific methods with which to perform inference. These methods typically fall into one of two major classes: frequentist approaches and Bayesian approaches. Within these are many subcategories, like nonparametric and resampling. Of course, regression problems involve an estimation of parameters, but we'll consider this category as the estimation of parameters outside of regressions. For example, if we fit a Student's t-copula to model the returns of two corporate bonds in a portfolio, how can we infer their correlation?

1.3.2 3.2 Simulating and Randomizing

This is one example where we do not need any data. Simulations provide an opportunity to create randomized data according to a known distribution. Generating the data is useful when we have new securities or histories that are too short or when we would like to test how our model works under known distributions. Monte Carlo (MC) simulations are the pre-eminent simulation in finance. We will use MC simulation to generate stock prices, returns, volatilities, interest rates, and many other variables. Depending on the complexity of the distribution, these simulations can provide a deep understanding of how models work with the specified type of data. Many methods in derivatives rely on MC simulations whenever simple closed-form analytic solutions do not exist. Recall that in Module 3, Lesson 2, we performed a Monte Carlo simulation to compute the Value at Risk from a Gaussian distribution.

If we want to impose a distribution on a system, then MC simulation tends to work very well. Other times, we may have a complex system in which it is implausible to run an MC simulation. In these cases, instead of working top down, we may work bottom up. One example of this approach is an agent-based model (ABM). Agent-based simulations are a different type of simulation that can model complex systems. Rather than fitting a distribution to a complex system with many interacting parts, an agent-based model (ABM) can be used to model the behavior of individual agents that then interact in complex ways. From this system, properties emerge that can create outcomes like those observed in the real world. Emergence is one of the key properties of ABM. We'll look at an ABM of a marketplace that replicates the Flash Crash.

In general, we'll use simulations for back-testing models; pricing and hedging derivatives; optimizing portfolios and trading strategies; and stress-testing risk models. In some cases, we'll generate multiple series of random variables, which preserve a given correlation to them. The ability to create random numbers is key throughout the entire program.

1.4 4. Conclusion

In this lesson and the next one, we are gathering and organizing the numerical methods you have encountered in this course and will encounter throughout the program. In this lesson, we focused on data cleaning, data engineering, and statistical analysis. Data cleaning identified imputing

missing values and excluding outliers. Data engineering focused on transforming and normalizing data, as well as methods for factoring and conditioning data. Statistical analysis covered inference and estimation, as well as simulating and randomizing. In the next lesson, we will complete the second half of numerical methods, including core modeling, model refinement, and finding optimal solutions.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Lesson 4 - Numerical Methods for Core Modeling, Model Refinement, and Finding Optimal Solutions

December 25, 2025

MODULE 7 | LESSON 4

1 Numerical Methods for Core Modeling, Model Refinement, and Finding Optimal Solutions

Reading Time	60 minutes
Prior Knowledge	basic mathematical foundations, Calculus, Linear Algebra, Probabilities, Statistics, basic programming concepts
Keywords	GARCH, PCA, Regression, Classification, Regularization, Modelling, Neural Networks, Ensemble, Calibration, Validation, Pseudocode, Hidden Markov, Filtering

In the previous lesson and this lesson, we organized and categorized numerical methods you will encounter throughout the program. In the previous lesson, we focused on data cleaning (imputation and outlier detection), data engineering (normalization and factoring), and statistical analysis (inference and simulation). In this next lesson, we will complete the second half of numerical methods, including core modeling, which will focus on regression, classification, dimensionality reduction, and conditioning; model refinement, which will focus on filtering and detecting change; and finding optimal solutions (including calibrating, optimizing, and combining models). Afterwards, we'll summarize a broader set of skills, including soft skills, that you will hone throughout the following eight courses.

1.1 1. Core Modeling

In this first section, we review two large groups of core models: regression/classification models and models that reduce the search space and/or condition of the data.

1.1.1 1.1 Regressing/Classifying

The most common technical skill is regressing one variable on various predictors. Those regressions can take many different forms, which we explore in Econometrics and Machine Learning. If the

target variable is continuous, we refer to the model as a regression problem. If the target variable is categorical, we refer to the model as a classification problem. We'll run linear and nonlinear regressions; cross-sectional and time series regressions; and univariate and multivariate regression. In some regressions, we'll easily observe the predictors; in others, we'll use factoring (another technical skill) to identify what the features are. Regressions can appear on the news channel, where you hear stories like "studies show that eating chocolate leads to a happier life" (undoubtedly sponsored by the chocolate companies).

From a software point of view, regressions are "black boxes" to which we supply inputs and crunch the numbers to produce the model outputs. However, like all the other areas, we will want to understand the details of the algorithm: its assumptions, derivation, advantages and disadvantages, and appropriate interpretations and applications of the results. In this program, black boxes should become white boxes, with a clear understanding of the mechanics to facilitate their best use.

In Financial Econometrics, you will encounter a very interesting regression problem for the variance rather than the level of a series. In a model known as GARCH, you will regress today's volatility using yesterday's volatility and yesterday's squared return. In Module 5, we cover generalized autoregressive conditional heteroskedasticity models, known as GARCH models, that are autoregression models for a time-varying volatility.

As you will see in Machine Learning, we will run a classification model using logistic regression. The difference between regression and classification mirrors the difference between continuous and categorical variables.

1.1.2 1.2 Reducing/Conditioning

The data we are given may be too big to handle. Learning how to divide the data is key. One method is using principal components: a data rotation technique that orders the data in orthogonal components ranked in order of decreasing variance. From this new set, we may choose only a selection of the components that captures the bulk of the variability of the data. Clustering is another way we may reduce the dimensionality of our data. Even when you perform conditional studies, you reduce the data by using only the subset that fulfills the condition. Classification trees and Naïve Bayes classifiers are two examples.

Recall from Module 1, Lesson 4 that a principal component analysis was done on n columns, and by keeping 3 principal components, we explained a vast majority of the variation of the data.

1.2 2. Model Refinement

In this section, we review two large groups of refining models: filtering and detecting change.

1.2.1 2.1 Filtering

Technically, filtering can happen anywhere in the data processing pipeline. For example, once we have imputed missing values and excluded outliers, we may want to filter data. We'll use filtering as a process to prepare data for analysis. At times, we may not need to do any filtering. End-of-day adjusted closing prices of an equity are likely ready to be converted to returns and modeled. However, an intraday version of the same stock series, with bid prices, ask prices, and trade prices, may require filtering to make the data more manageable.

Within a model, one way to think of filtering is applying weights. (Assigning a weight of 0 would mean we are excluding.) Other times, we will use filters to smooth data due to high noise content. One example of filtering will be the Kalman filter for improved estimation of signals. We'll see these mathematical topics in Stochastic Modeling. Another type of filtering is regularization, where we effectively assign penalties to complexity.

For example, in the Financial Econometrics course, we explore regularization techniques such as L1 (Lasso) and L2 (Ridge) when working with Linear Regression models. These methods help us “filter out” less important variables (in the case of Lasso) or normalize the distribution of regression coefficients by assigning weights in a more robust way (in the case of Ridge). These powerful techniques are just a glimpse of what you'll learn in Financial Econometrics, where we dive deeper into sophisticated statistical methods for analyzing financial data.

1.2.2 2.2 Detecting Change

One of the most useful technical skills in statistics is the ability to detect meaningful changes in the system. Detecting change may trigger a trading model to start trading or a risk model to no longer be effective due to exceeding a volatility breach. Changes can occur statistically when we look at a process that transitions from stationary to non-stationary or from mean-reverting to trending or that switches signs from positive to negative. We'll explore change detection when we run tests for unit roots and stationarity. We'll explore these in regime shifting models. We'll look at error correction models within this group to determine how a temporary disequilibrium may revert to their equilibrium.

Furthermore, we can think of detecting change as a rate of change, which brings us to derivatives. Classical calculus teaches us that the first derivative represents the rate of change. We'll learn methods of stochastic calculus and learn how to take derivatives, including partial derivatives, of stochastic functions. These derivatives will turn out to be key sensitivities of how an option price varies with the underlying variables and parameters.

In our Stochastic Modeling course, you'll discover fascinating tools like Hidden Markov Models (HMM) that help us understand how financial markets switch between different states or “regimes.” You will learn how to detect when the market transitions from a low-volatility “calm” state to a high-volatility “turbulent” state, or from a bullish to a bearish regime. This powerful technique allows analysts to adapt their strategies based on these changing market conditions. HMMs are just one example of the sophisticated yet practical tools you'll master in Stochastic Modeling.

1.3 3. Finding Optimal Solutions

In this section, we review two large groups of refining models: calibrating/optimizing and combining models.

1.3.1 3.1 Calibrating/Optimizing: Finding Solutions

Sometimes, we assume values for the constant parameters of models and then use them to price securities or estimate random variables. Other times, we will want to work in the opposite direction: input the prices and value from the current market and imply what parameter values cause those prices. This process is known as calibration. Optimization is related because it seeks to find a minimum (if maximum, we take the negative) of an objective function by changing the value of key parameters subject to specified constraints. Optimizations are a type of constrained regression

if you will. These tools will require us to be relentlessly methodical and meticulous in our ability to hunt for the global minimum. Thanks to an ensemble of topics in stochastic modeling, machine learning, and deep learning, you will encounter many examples of both calibration and optimization. One of the great quants, the late Peter Carr, said that the top skill the industry looks for from FE graduates is their ability to calibrate and optimize models they use. The skills required range across mathematics, statistics, and programming so that deficiencies in any one area limit the optimizer in performing their job. For example, we'll estimate the coefficients of a GARCH(1,1) model using Bayesian techniques.

Optimizations appear in many places. Every regression problem has an associated optimization problem (minimize the sum of least squares; maximize the likelihood, etc.). Many imputations will involve an optimization. When we train and test a model, we will want to optimize the bias-variance tradeoff by minimizing a mean squared error or similar metric. In Module 5 of the following Financial Econometrics course, we will learn the Bayesian technique to estimate a GARCH model. Also, in Module 2 of the Stochastic Modeling course, we will learn how to use various methods to calibrate a credit risk model.

1.3.2 3.2 Combining: Strength in Numbers

Sometimes, the “best” model is not a single model but a combination of models. Indeed, we will be examining how models that were developed independently can actually be combined to work better. For example, in a future course, you will see how an econometric model GARCH can be combined with a deep learning model to improve its performance. Instead of always looking for the best model, we may combine models. For instance, in Machine Learning, you will see two linear regression methods, one called LASSO and the other called Ridge regression. Which works better? Well, there's a third method, called elastic nets, that uses both in the same regression. So oftentimes, you may use elastic nets and set the weights to include some amount of each model. In addition, we'll learn how to combine results from different types of models through ensemble learning. Specifically, we'll discuss bumping, bagging, and especially boosting. As we increase our modeling ability, we'll even be able to combine classical models like GARCH with machine learning algorithms. For example, we can combine GARCH models and neural networks to get better results than with either one individually. In the following Machine Learning course, you will learn how to combine results from several models to produce better predictions than a single model in Module 4. This module will provide us more information on this topic.

1.4 4. Building Your Technical Foundation

1.4.1 4.1 Mathematics

The Quantitative Proficiency Test addressed many of the mathematical areas. Math provides much needed rigor. Often, engineering students may feel that the code or solution is enough: a problem of the ends justifying the means. The problem is that mathematics is the language that is needed to get to the heart of understanding. It is not essential to master the math as if you were a mathematician. Indeed, math is rigorous in that the mathematician is tasked with proofs of theorem, lemmas, and many details. The financial engineer (FE) needs to learn enough math to be rigorous when needed but not too much at the expense of other areas. Math is almost never learned by reading the book but by reading, rereading, getting used to notation, studying examples, solving problems, getting stuck, figuring out where the disconnect occurs, remedying that, and repeating. Math skills also include the ability to read and create equations in LaTeX so that you can post or

reply in forums and complete your assignments correctly.

1.4.2 4.2 Modeling: Statistics/Machine Learning

Continuing our discussion of the quantitative area, let's call modeling skills a separate skill set. Under the umbrella of mathematics, we have linear algebra, calculus, probability, and discrete mathematics. In these areas, questions are more objective. There are more theorems, transformations, and processes than there are models, p-values, and test scores. Linear regression requires an understanding of mathematics (e.g., expressing the equation in matrix notation or implementing a maximum likelihood estimation). But linear regression is also an art. Two different statisticians working with the same data may come up with two different looking models that perform equally well. Some models may favor an easier interpretation, others a good prediction rate, and others could help by making costly errors less likely. We'll develop metrics for all these models starting in Econometrics and continuing in Machine Learning. There are often many ways to derive factors, use them in regressions, and combine the results. Statistics and ML are empirical and require an understanding of the assumption (often expressed mathematically) behind the models and this use.

1.4.3 4.3 Coding Skills

Being skilled in the mathematical and modeling areas, one may think the rest of the work is easy. This assumption is not the case because of implementation. Unfortunately, many students do not get enough hands-on experience coding. Coding involves multiple skills. Let's divide these into four areas.

The first skill is problem solving. Can students tackle a project by writing pseudocode that solves the problem devoid of any language, architecture, or framework? It is stunning that some students cannot outline clear instructions for an algorithm when taking a quiz, completing a capstone project, or answering an interview question. To experienced people, this step of an ordered, comprehensive set of instructions is second nature and is not explicitly separate from the coding. Nevertheless, as you learn this material, make sure you understand an algorithm, transformation, or model by outlining its steps in a spoken language (e.g., English) rather than Python.

The second skill is design. How will students write functions, design classes, and apply design patterns to architect a solution? For a simple project, writing a handful of functions may work well enough. But as a project's complexity and scope increase, it is prudent to design how the solution looks. While a certificate in software design or architecture is certainly not needed, an FE engineers a solution in software, so that solution should be organized to solve the problem efficiently, with room to make changes and additions. Practical experience shows that more time can be spent maintaining code than writing it in the first place. So solid code should allow the FE to maintain and update code as needed.

The third skill is the actual coding itself. Our official coding language is Python. That means you might learn how to do things in Python the same way you would have done them in C# or Java. Alternatively, you might learn how to do things the Pythonic way. These skills run the gamut of importing and exporting data, working with the right data structures, managing memory, and optimizing your use of existing packages. By the end of the program, Python should become as familiar as typing. You will literally spend hundreds of hours coding throughout the curriculum, including running existing examples, modifying and commenting code, and writing your own functions, classes, and even a package.

A fourth skill is debugging. It's amazing how much code is written without any checks. Students have submitted assignments with negative stock prices, negative probabilities, and negative volatilities. Some have submitted simulations with stock prices that don't change or where they climb to 10 to the 300th power! Debugging is essential because even if you can write code perfectly the first time, you are likely going to review a colleague's code at some point. Comments may be outdated if they even exist at all. There are often no help files, or the original author may be inaccessible (e.g., for model validation teams) or completely unavailable if they are not with the firm anymore. Debugging is more than throwing print statements in the code. It is a systematic way to find and fix errors as efficiently as possible. While the goal of this program is NOT to make you full-time software engineers, as an FE, you engineer models in code, so bugs in the code reflect on your skill as an FE. Remember that a weakness in coding due to a bug shows up as an FE who has not mastered their model.

1.5 5. Focus on Finance

Think of a stool with 3 legs: the legs are mathematics, statistics/machine learning, and computation. Strengthening each leg ensures the stool is sturdy. With a balanced, strong set of legs, the stool provides a place to sit at the table. On top of the table is a finance problem. Focus on understanding the problem from a financial point of view. When you enjoy and are successful at technical problems, it is tempting to focus on the technicalities, be attracted to learning new material, and take pride in the craftsmanship. These are noble attributes, and to be sure, the details are important.

But rather than focus on the tools, the emphasis should be on the value your tools bring to solving a problem. For example, suppose volatility is being measured with a metric X. You are excited to learn a new area, so you spend a few weeks using tools from that area to build a new metric Y. In the end, your new metric Y measures volatility very much the same as before. It is not more insightful, not quicker, not cheaper, not better—just the same. What is the value in such a project? There are liberties to undertake new research areas, but imagine project after project where the emphasis is on the tools and not the problems. If the project were motivated by a need to produce the metric more quickly, more accurately, or with greater insight, then the focus is finance driven rather than tool driven. Perhaps a metric is needed that works in sideways markets better. Perhaps a metric is needed on days where the trading volume is well above a certain threshold. This focus is key because it reminds us that we are engineering solutions to problems and not merely coding for coding's sake or doing math for math's sake. Maintain an element of practicality.

1.6 6. Building Your Non-Technical Foundation

Technical skills alone will not allow you to collaborate and communicate effectively with others. These non-technical skills are essential for success in working with others.

1.6.1 6.1 Communication Skills

Communication skills encompass a broad set of areas: visualizations, presentations, speaking, and written skills. Equipped with a trisection of math, modeling, and computational skills, students may think they are well prepared to enter the industry. However, there is another array of skills that is essential for professional success. Collectively, these are sometimes known as soft skills. For some people they are easier, and for others, they are much more difficult. The first is communication.

Our program builds communication skills through assignments, projects, and occasional presentations. The first of these areas is in visualization. How effectively can you communicate the results or application of your model? For example, some students print two pages of numerical output in an assignment thinking that is thorough. Why print out hundreds of numbers when a time series graph, histogram, or summary table will convey the same ideas in a cleaner, clearer, and more concise manner?

Although FE is a technical area, professionals are expected to communicate ideas to both technical and non-technical audiences. This includes your ability to write documents in which you keep your target audience in mind. Presenting to an audience is another important skill, whether it's an in-person or virtual presentation. Preparing a slide deck is a communication skill that finds a balance between the details in a project and the limited time and space to emphasize key points, some of which can be technical. Presentations can vary from code reviews to conferences, weekly team meetings, or even a milestone presentation given to senior management. Being able to use language, tone, and details effectively is an important skill. Managers tend to prefer employees who give effective presentations because they trust these people will be able to represent the group in a positive light to both internal colleagues and external clients. Fortunately, you will take a workshop on giving presentations. Interestingly, your technical skills can certainly assist in your ability to give effective presentations. Likewise, mastering a technical area should mean that you are able to explain the ideas and applications to both general audiences and technical audiences.

Communication also involves reading and listening. This includes being able to read forum posts, instructions for an assignment, and different types of required readings in your classes. You will also practice communicating with your fellow group members on more than two dozen group work projects throughout your coursework. Let's call those collaboration skills, the topic of the next section.

1.6.2 6.2 Collaboration Skills

Most working professionals must learn to collaborate. Few and far between are the jobs where one person works in isolation. Oftentimes, a qualified individual is not hired because people feel they might be difficult to work with. Learning to work with others is not always easy, especially if there is a difference in motivation, backgrounds, the time each member is willing to commit, the quality of coding, etc. Even with a team of talented individuals, personality differences and egos can get in the way. The opportunity to work with fellow classmates exists throughout the program. Each course contains three different projects where you will work with no more than three people. The group work projects are intended not only to have you master the material but to also learn to collaborate with others.

There are different ways groups collaborate. Sadly, some groups will have a negative experience: one or two people do all the work and the third member makes excuses and submits no work. Usually, the non-participating member doesn't inform the other members until hours before the deadline, leaving their groupmates scrambling to get the work done. Other groups engage in a divide-and-conquer strategy: "I do this, you do that, then we combine the work and submit." There is no group effort, no discussion, no brainstorming, and no checks and balances.

Neither of these scenarios is our intention. The assignments are structured so that group members have some individual accountability but also must work as a team. Effective groups are teams where ideas can be freely exchanged, questions get answered, new questions get asked, and the results are better than any one member could have done individually. Good group members will

become colleagues and friends. Take this opportunity to make the most of your group work by collaborating in a timely, respectful, and effective manner. Throughout years of teaching in this program, instructors have been amazed at the quality of some group work and disappointed at the missed opportunities of other groups.

1.6.3 6.3 Critical Thinking Skills

Our previous lesson addressed model failure and crises. As a student in the program, you might ask, “Knowing models fail and crises occur, what do I need to do differently?” The answer is to think critically.

It is likely that critical thinking is the skill that employers value most. It means that their employee is a growth stock, capable of learning, investigating, synthesizing, re-evaluating, questioning, and recommending. Throughout your quizzes, collaborative review tasks, group work projects, and other assignments, you will be challenged to develop critical thinking skills. These include:

- Identifying Disruptions: Examining the impact central banks, regulatory agencies, and FinTech companies have on market volatility, liquidity, and profitability.

- Detecting Biases: Identifying behavioral biases that affect asset pricing, asymmetries in markets, and risk regulation.

- Systems Thinking: Viewing global markets, exchanges, and risk regulation in a systems framework.

- Researching: Collecting information such as market data, regulatory policies, and tax laws to conduct research.

- Problem-Solving: Categorizing financial engineering problems as pricing, hedging, or optimizing perspectives so that existing solution methodologies can be tried, as well as innovative solutions.

- Interpreting: Interpreting the sensitivities (e.g., Greeks in derivatives) and market implications of parameter values from various statistical and machine learning models considering financial data.

- Stressing: Reflecting on worst-case scenarios by stress testing existing models using crisis data and scenario analyses.

- Critiquing: Critiquing the limitations of models that no longer work in certain economic cycles, market conditions, or financial crises.

- Analyzing: Conducting a transaction analysis of an investment offering or business venture through a simplifying analysis of its hedging strategy.

- Recommending: Recommending strategic portfolio or risk allocations based upon interpreting the results from various models.

1.7 7. Conclusion

If you only have the mindset of a mathematician, you simply write equations, derivations, and solutions on paper. If you just have the mindset of a programmer, you write the code, compile it, and feel a sense of accomplishment when it runs. If you only have the mindset of a statistician, you run the data through the model and call it quits when there's output. An FE needs to build the mindset of all three of those while also connecting the dots: how does this metric, model, or methodology solve a problem that was yet unsolved, improve a solution we already have, or give insight to a process that did not exist? Focus on the financial problem.

Copyright 2024 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.